8

Elements of Control Theory

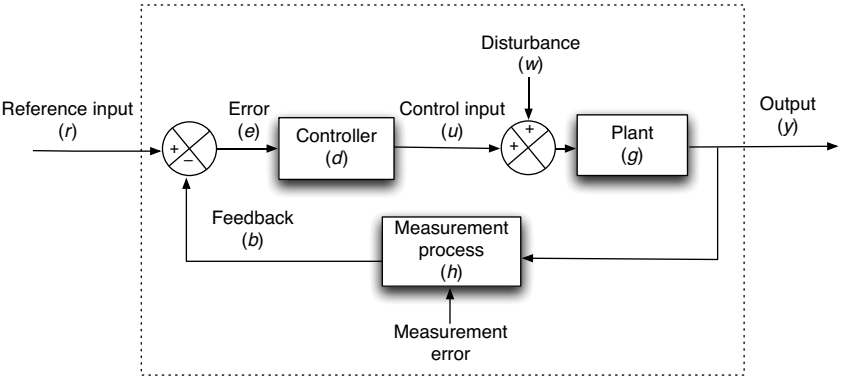
A computer network is an engineered system designed to meet certain design goals. Two such goals are **responsiveness** and **stability**. Responsiveness allows a system to continue to meet its performance goals despite changes in its operating environment, such as workload fluctuations or component failures. Stability prevents a system from failing or from being sent into uncontrollable oscillations due to certain inputs. Control theory provides powerful tools for building responsive and stable systems. For example, mathematically modeling the behavior of a controlled system demonstrates a fundamental contradiction between responsiveness and stability: The more responsive a system, the greater the chance that it will be unstable.

This chapter presents the elements of control theory. For simplicity, we consider primarily continuous-time linear control systems and classical control strategies. The concepts here, however, should provide a strong foundation for the interested student to delve further into this subject. Note that Chapter 5 is an essential prerequisite for understanding the material in this chapter.

We first describe a generic control system. We then characterize the behavior of generic first- and second-order systems, culminating with the design and analysis of classical control systems. We end with an introduction to "modern" control systems: state space representation and the principles of observability and controllability, and digital control.

8.1 Overview of a Controlled System

A generic feedback control system is shown in Figure 8.1. The system to be controlled is usually referred to as the **plant**. We assume that the plant and all control elements are continuous, linear, and time invariant. Given a control **input signal** u, the plant responds with an **output signal** y that is also additively influenced by an uncontrollable **disturbance signal** w. The control goal is to maintain the output of the plant at some **reference value** or **command** r despite disturbances. This is done using a **controller**, which compares the reference signal value with the **measured output signal** b, where this measurement itself may be subject to error. The controller chooses a **control signal** u such that the future output of the plant eventually matches the reference input, despite disturbances and measurement errors.



11:05

Figure 8.1 General model of a controlled system

EXAMPLE 8.1: A SIMPLE CONTROL SYSTEM

A thermostat-controlled home-heating furnace is an example of a simple control system. A user can use the control interface to specify a desired home temperature r. The controller compares this temperature with the temperature b measured using a sensor. If the difference (error e) is positive, the controller activates a heating furnace by sending it a control signal u, eventually raising the home's air temperature y to the desired reference value. At this point, the error drops to zero and the furnace is turned off.[1] A disturbance w, such as the

1. This example is highly simplified. Modern furnaces also have more sophisticated control elements, such as a controller that turns on the furnace slightly before it is needed, so that it heats air instead of accidentally cooling it, and a controller that uses measurements to predict how long a furnace needs to be on to heat the home by a certain amount.

inrush of cold outside air due to opening a door or a window, results in a drop of the measured temperature y. The controller compensates for this disturbance by turning on the furnace and eventually causing the output to return to the reference temperature.

Even this simple example demonstrates three lessons that apply to all control systems. First, immediately after the disturbance, a home's temperature could be far from the reference temperature. The presence of the controller does not guarantee that the home temperature will always be at the reference value, only that, in the absence of disturbances, this reference value will eventually be reached. If a series of disturbances come in quick succession, the system may never reach equilibrium.

Second, given that a system is currently not in equilibrium, it is up to the controller to decide how quickly to restore the system to this state. A system is said to have a high **gain**, or be **responsive**, if the controller tries to restore the system rapidly. Although this is usually desirable, a system with too high a gain can become unstable because in such a system, even a short delay in transferring measurements from the plant to the controller causes the controller to overreact to stale information, causing oscillations. In this example, a controller with too high a gain could result in a home whose temperature persistently oscillates from being too hot to being too cold. Choosing the degree of gain in the controller that balances responsiveness and stability is a critical issue in any control system.

Third, note that the controller simply cannot compensate for measurement errors. For example, if the temperature sensor always reads two degrees higher than the true temperature, the home's temperature will always be two degrees lower than the reference value. Therefore, in any feedback control system, it is important to use the best sensors possible.

EXAMPLE 8.2: CONTROLLING WEB SERVER LOAD

Consider a Web server that responds to GET and POST queries from Web browsers. The server uses a buffer to hold pending requests before they are served so that requests arriving to a busy server are not lost. Figure 8.2(a) shows the times at which requests arrive and depart from the buffer. Requests arrive at discrete time instants (shown on the X-axis) and are served after spending some time in the buffer. For example, request 4 arrives at time 5 and is served at time 7. The system is causal, so the departure curve is always at or below the arrival curve.

Although arrivals and departures are discrete events, it is convenient to model them using a continuous "fluid" approximation as shown in Figure 8.2(b).

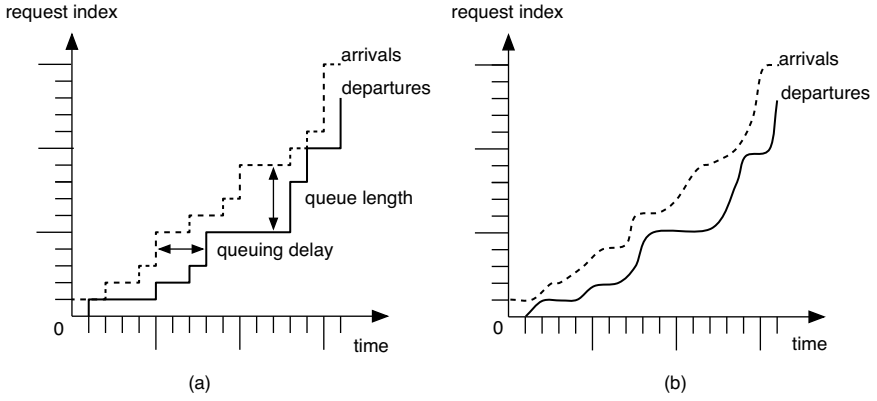


Figure 8.2 (a) Discrete arrivals and departures; (b) fluid approximation

Intuitively, this approximation is obtained by smoothing the arrival and departure curves, which removes abrupt transitions without introducing too much distortion. If queries arrive quickly enough, the fluid approximation has been found in practice to be reasonably accurate and, in any case, greatly simplifies mathematical analysis.

Note that the vertical distance between the arrival and departure curves at any point in time represents the number of queries in the buffer (the queue length) at that time. The control goal is to ensure that this quantity neither exceeds the buffer capacity—causing requests to be lost owing to buffer overflow—nor drops to zero—causing a buffer underflow and idling the Web server, unless, of course, there is no work to be done. Instead, we would like to keep the number of queries in the buffer at some reference value r that is small enough to not greatly impact the server response time but large enough that if the Web server were to suddenly obtain additional capacity, the buffer would not underflow. The greater the value of r, the higher the expected query service rate and the higher the query-processing delay.

The problem is that a client does not know the Web server's service rate, which depends on how many other clients are being served and how long it takes to satisfy a client request. We model this uncertainty by representing the Web server's service rate as a time-varying "disturbance" $w(t)$. The control goal is to manage the client request rate $u(t)$ to prevent buffer underflow and overflow. Specifically, we assume that the controller is given a buffer-occupancy reference value $r(t)$ and that it is instantaneously informed of the current number of bits in the buffer, denoted $y(t)$. The controller uses this feedback information to choose $u(t)$ such that $y(t)$ tracks $r(t)$ despite disturbance $w(t)$.

Note the similarity between this control system and a home thermostat. In both cases, the controller tries to keep output y at the reference level r despite disturbance w by comparing the measured state of the system with the desired reference value.

8.2 Modeling a System

Motivated by Example 8.2, we turn our attention to the process of mathematically modeling a system, which can be viewed either as the plant or as the controlled system shown in the dotted box in Figure 8.1. Both the plant and the controlled system have an input and an output, where the output depends on the input. By using identical approaches to model both the *open* uncontrolled system and the *closed* controlled system, we can recursively control a complex system. That is, we can model and control one aspect of a plant, then represent this controlled system as black box when modeling the remainder of the system. This approach is necessary to control a complex system. We will study some practical problems using this approach in Section 8.7.

Choosing which aspects of a system should be modeled and which aspects can be ignored is an art rather than a science. Developing a system model that is mathematically tractable and yet represents all the relevant aspects of the system will usually require several rounds of refinement. We first outline an approach that has been found to work well in practice and then focus on three mathematically equivalent representations of a system.

8.2.1 Modeling Approach

Start by understanding the working of the system and its component parts. Determine the physical capabilities of each component, its limits of operation, and the operating regimes where it is linear or nonlinear. Components in a control system can usually be described in terms of the concepts of **effort** and **flow**. Effort represents a system input that causes a flow. The product of effort and flow is **power**, and the total power expended over some time duration is called **energy**. For example, in a mechanical system, forces correspond to effort, and velocities correspond to flow. The product of force and velocity is mechanical power, and the integral of power over time is the amount of mechanical energy used. In an electrical system, voltages correspond to effort and currents to flow. Their product is electrical power, whose integral is electrical energy. Similarly, in a computer network, the quantity of data that needs to be sent from a source to a destination represents effort, and the rate at which data is sent corresponds to flow. Although they can be analogously

defined, the concepts of power and energy are not commonly used in computer networks.

System components can be either active or passive. Active elements generate power (not energy!) that flows through the system. The dynamic operation of the system is essentially an attempt to redistribute this power over time. In contrast, passive elements store and dissipate energy but do not generate power. In an electrical system, a voltage source, such as a battery, is an active element that drives the system, whereas capacitors and inductors are passive elements that store and dissipate energy. Similarly, a data source is an active component that generates effort that causes a flow on links and into routers. In contrast, a buffer is a passive component that stores and releases energy, like a capacitor.

The next step is to capture the laws, such as conservation, that constrain the operation of the system. For example, in a mechanical system, Newton's laws form the basic equilibrium constraints. In a computer network, the rate at which a buffer stores data is the difference between the data arrival and departure rates. Similarly, during the slow-start phase of a TCP connection, the rate of transmission grows exponentially. These constitute the mathematical laws of operation of the corresponding components.

Finally, state the intended goal of the control system in terms of the ideal equilibrium operating point (which, by convention, is defined to be zero). If possible, quantify the permissible operating error. This makes it possible to determine the degree to which a chosen control law meets requirements.

At the end of the modeling process, it should be possible to capture the operation of the system in a handful of equations. We will see some concrete examples of the modeling process shortly.

8.2.2 Mathematical Representation

Three common ways to mathematically represent a system (i.e., either the plant or the dotted box in Figure 8.1) are by the **state variable**, the **impulse response**, and the **transfer function** representations. Our discussion assumes that the system is **linear** and **time invariant** (**LTI**). Recall that a time-invariant system is one whose parameters do not change with time: If the system has an output $y(t)$ at time t for an input $u(t)$, it has the output $y(t - T)$ for an input $u(t - T)$. That is, if $y(t) = G(u(t))$, then $y(t - T) = G(u(t - T))$. Recall also that a linear system is one that exhibits **superposition**: If the input u_1 leads to output y_1 and input u_2 leads to output y_2, then for all constants k_1 and k_2, the input $(k_1u_1 + k_2u_2)$ leads to the output $(k_1y_1 + k_2y_2)$. That is, if $y_1 = G(u_1)$ and $y_2 = G(u_2)$, then $k_1y_1 + k_2y_2 = G(k_1u_1 + k_2u_2)$. For an LTI system, it is always possible to choose the initial time to be $t = 0$. We assume the system to be causal, so the input signal is zero for $t < 0$.

13:56

State Variable Representation. In the state variable representation, we first choose a set of state variables that completely characterize the state of the system. Note that by choosing the set of state variables, we are implicitly deciding which aspects of a system to model and which to ignore: a subtle and far-reaching decision.

The number of state variables in the system typically corresponds to the number of energy-storage elements in the system, because each state variable tracks the amount of energy currently in that storage element. In an electrical network with, say, three capacitors and four inductors, we need at least seven state variables. For the Web server in Example 8.2, there is only one energy-storage element, so we need only one state variable: for example, the buffer-occupancy level $y(t)$, which, in this simple example, is also its output variable. In general, the set of state variables is not unique: The system could equally well be characterized by a state variable corresponding to $\dot{y}(t)$, which is the buffer fill or drain rate rather than its occupancy level.

We denote the set of state variables by the column vector $\boldsymbol{x}(t)$, whose dimension is the **order** of the system. The system is represented by the equations that govern the evolution of $\boldsymbol{x}(t)$. We first consider a **single-input single-output system**, with scalar input u and a scalar output y. To model the complete system (i.e., including the disturbance input), we also consider a vector disturbance $\boldsymbol{w}$. Let

$$\boldsymbol{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \dots \\ x_n(t) \end{bmatrix}$$

Then, any linear time-invariant system can be represented using the following state-update equations, where the parameter t has been left out for clarity:

$$\begin{aligned} \dot{x_1} &= a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n + b_1u + f_1w_1 \\ \dot{x_2} &= a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n + b_2u + f_2w_2 \\ &\dots \\ \dot{x_n} &= a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n + b_nu + f_nw_n \end{aligned} \qquad \textbf{(EQ 8.1)}$$

Moreover, the output can be written as a function of the state, the disturbances, and the input as

$$y(t) = c_1x_1 + c_2x_2 + \dots + c_nx_n + du + e_1w_1 + \dots + e_nw_n$$

where scalar constants are given by a_i, b_i, c_i, e_i, and f_i. Using matrix notation, we can compactly write this as

$$\begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}u + \boldsymbol{F}\boldsymbol{w} \\ y &= \boldsymbol{c}^T\boldsymbol{x} + du + \boldsymbol{e}^T\boldsymbol{w} \end{aligned} \quad \text{(EQ 8.2)}$$

where the matrix $\boldsymbol{A}$, the diagonal matrix $\boldsymbol{F}$, and the column vectors $\boldsymbol{b}$, $\boldsymbol{c}$, $\boldsymbol{w}$, and $\boldsymbol{e}$ are appropriately defined.

Note that the a_{ii} terms represent how the state element x_i evolves independent of the other state elements, whereas the a_{ij} terms represent the degree of coupling between the ith and jth state elements. If these coupling terms are zero, the matrix $\boldsymbol{A}$ is diagonal, and the system can be decomposed into a set of n noninteracting systems. It is often possible to transform the basis set of state elements into an alternative set of elements that either diagonalizes $\boldsymbol{A}$ or at least puts it in the Jordan normal form, with elements only on the diagonal and one row above. This maximally decouples the system, making it easier to analyze. This **similarity transformation** is discussed further in Section 3.5.7.

EXAMPLE 8.3: STATE REPRESENTATION OF A WEB SERVER

We use the state space representation to model the Web server of Example 8.2. We wish to model the entire system, so the model incorporates the disturbance input w. We choose the state variable $x(t)$ to be the same as the buffer-occupancy level $y(t)$. Then,

$$\dot{x} = u - w$$

is the state-update equation of the system. Clearly, the output of the system is identical to the buffer-occupancy level $y(t)$.

The state space model can be easily extended in two ways. First, it can be used to represent systems with multiple inputs and outputs. In this case, the input u is replaced by the vector $\boldsymbol{u}$, and the output y is replaced by the vector $\boldsymbol{y}$. The details are straightforward and are left as an exercise to the reader.

Second, the state space model can also be used to represent nonlinear time-invariant systems as follows:

$$\begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{w}) \\ \boldsymbol{y} &= \boldsymbol{g}(\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{w}) \end{aligned} \quad \text{(EQ 8.3)}$$

where the vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ represent the state and output vectors of a multiple-input multiple-output system with input vector $\boldsymbol{u}$ and disturbance vector $\boldsymbol{w}$, and the vector functions $\boldsymbol{f}$ and $\boldsymbol{g}$ represent arbitrary functions. It is often possible to **linearize**

such a system around an operating point $\boldsymbol{x_0}$, $\boldsymbol{u_0}$, $\boldsymbol{w_0}$ by approximating the functions $\boldsymbol{f}$ and $\boldsymbol{g}$, using the multivariable form of the **Taylor expansion**:

$$\dot{\boldsymbol{x}} \approx \boldsymbol{f}(\boldsymbol{x}_0, \boldsymbol{u}_0, \boldsymbol{w}_0) + (\boldsymbol{x} - \boldsymbol{x}_0)^T \nabla x \boldsymbol{f} + (\boldsymbol{u} - \boldsymbol{u}_0)^T \nabla u \boldsymbol{f} + (\boldsymbol{w} - \boldsymbol{w}_0)^T \nabla w \boldsymbol{f} \qquad \textbf{(EQ 8.4)}$$

where $\nabla x \boldsymbol{f}$, $\nabla u \boldsymbol{f}$, and $\nabla w \boldsymbol{f}$ are the gradients of $\boldsymbol{f}$ with respect to the $\boldsymbol{x}$, $\boldsymbol{u}$, and $\boldsymbol{w}$ vectors, respectively and the products are Hadamard products i.e. row-wise multiplication. The function $\boldsymbol{g}$ can be similarly linearized. Note that this approximate linearization is valid only in the immediate neighborhood of the equi- librium operating point $\boldsymbol{x}$**0**.

Impulse Response Model. Many practical systems of interest have a single scalar input u. In this case, it is possible to represent the system by its **impulse response**. Recall from Section 5.4.3 that if we know the response $g(t)$ of an LTI system to a Dirac delta input, we can compute its response to *any* input $u(t)$ by convolving the input signal $u(t)$ with the impulse response $g(t)$. When using this model, we assume that the system is in equilibrium at time t=0, or $\boldsymbol{x}(0) = \boldsymbol{0}$. Also, if we are modeling the entire system (not just the plant), the input signal should incorporate the disturbance. The output $y(t)$ is given by

$$y(t) = \int_0^t u(\tau) g(t - \tau) d\tau = u(t) \otimes g(t) \qquad \textbf{(EQ 8.5)}$$

The impulse response model is rarely used in practice: Its main purpose is as a step in the derivation of the *transfer function model*, which we describe next.

Transfer Function Model. Consider a system at equilibrium at time 0 that is described by the impulse response function $g(t)$, so that the output $y(t)$ is given by Equation 8.5. By taking the Laplace transform of both sides we get:


9:45

$$Y(s) = U(s)G(s) \qquad \textbf{(EQ 8.6)}$$

In other words, we obtain the Laplace transform of the output by multiplying the Laplace transform of the input (which, if necessary, incorporates a disturbance input) with the Laplace transform of the impulse response, rather than computing a convolution, as is required with the impulse response function. This simple form of the output makes it convenient to represent a single-input single-output causal system by the Laplace transform of its impulse response, which we call its **transfer function**.

The transfer function of many common systems takes the form $N(s)/D(s)$, where $N(s)$ and $D(s)$ are polynomials in s. The highest power of s in $D(s)$ is called the **order** of the transfer function. Consider a denominator polynomial $D(s)$ of order m. The m roots of $D(s)$, that is, the m values of s for which $D(s)$ is zero, denoted $\alpha_1, \alpha_2, \ldots, \alpha_m$, are the values of s for which $Y(s)$ is undefined, and these are called

the **poles** of the system. It turns out that these poles have a deep significance because, using the technique of partial fraction expansion discussed in Section 8.11, $Y(s)$ can be written as a sum of terms of the form $\frac{A}{(s-\alpha)^r}$. Referring to row 6 of Table 5.4 in Section 5.8 and recalling that the Laplace transform is linear, we see that the corresponding term in $y(t)$, obtained by taking the inverse Laplace transform, is $\frac{At^{r-1}}{(r-1)!}e^{\alpha t}$. Therefore, if the pole is negative, the corresponding term in the output decays with time; if, on the other hand, it is positive, the output grows with time. We like a system to be **stable**, which informally means that for a bounded input, the output also should be bounded. This is possible only if all system poles lie in the left half of the complex s plane. (We discuss this notion in more detail in Section 8.8.) Thus, the nature of $D(s)$ allows us to quickly determine the overall behavior of the system, reinforcing the usefulness of the transfer function system model.

EXAMPLE 8.4: TRANSFER FUNCTION MODEL

We now derive the transfer function model of the Web server system. Recall that the state space representation of the system is

$$\dot{x} = u - w$$

This is a single-input single-output system that is initially at equilibrium, so that $y(0) = 0$. Therefore, we can use a transfer function to model it. The output y is the same as the state variable x. To incorporate the disturbance into the input signal, we write

$$v = u - w$$
$$\dot{x} = v$$

We obtain the transfer function by taking the unilateral Laplace transform of both sides:

$$sY(s) - y(0) = V(s)$$
$$Y(s) = V(s)\left(\frac{1}{s}\right) + \frac{y(0)}{s}$$

so that

$$G(s) = \frac{1}{s}$$

is the required transfer function model of the system.

"Modern" versus "Classical" Control. The classical approach to control uses transfer function models. Although the results are intuitive, this approach does not permit the modeling of multiple-input multiple-output and nonlinear systems. The "modern" approach, instead, uses the state variable model, which easily generalizes to these domains and can also draw on linear algebra for compact notation and a deep theoretical basis. In keeping with our goal of presenting intuition rather than formalisms, we will use the classical approach, with a brief treatment of the state variable approach in Section 8.8.

8.3 A First-Order System

8:29

One of the simplest systems that can be characterized by a transfer function is a **first-order** system with a transfer function

$$G(s) = \frac{K}{1+\tau s} \qquad \text{(EQ 8.7)}$$

where K is called the **gain parameter** and τ is called the **system time constant**. The reason for this nomenclature is best understood by considering the response of this system to a step input $u(t)$, that is, an input that is one unit for all $t > 0$. Recall that the Laplace transform of a step input is $1/s$. Therefore, the Laplace transform of the output, $Y(s)$, is given by

$$Y(s) = \frac{K}{s(1+\tau s)}$$

It is easy to see that the roots of the denominator are s=0 and $s = -\frac{1}{\tau}$. We use this fact to compute the partial fraction expansion as

$$Y(s) = K\left[\frac{1}{s} - \frac{\tau}{1+\tau s}\right]$$

Using Table 5.4 in Section 5.8, we compute

$$y(t) = K\left(1 - e^{\frac{-t}{\tau}}\right) \qquad \text{(EQ 8.8)}$$

which is shown in Figure 8.3 for $K = 2.5$ and $\tau = 2$.

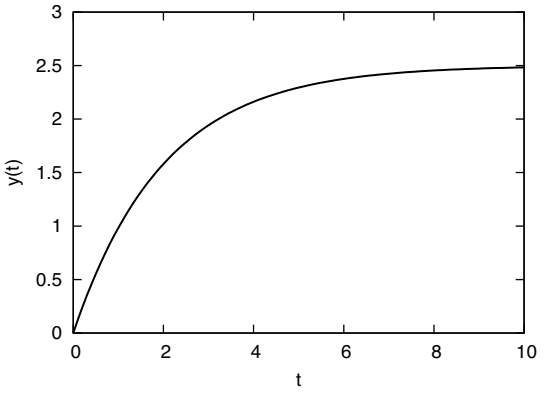


Figure 8.3 Step response of a first-order system with gain 2.5 and time constant 2

A close examination of Equation 8.8 and Figure 8.3 reveals the following.

- In response to a step input, the output rises rapidly, eventually asymptoting to a value that is K times the input value of 1. In other words, in steady state, the system magnifies the input by a factor of K, thus justifying K as the system gain parameter.
- The system reaches $(1 - 1/e)$, or roughly 63% of its final value, when $t = \tau = 2$ and 99.33% of its final value when $t = 5\tau = 10$. For all practical purposes, the system reaches its asymptotic value at time 5τ, justifying τ as the system time constant.
- A step input is like a sharp kick to the system. We can write the system's **step response** to this input as $y(t) = K(1 - e^{-t/\tau}) = K - Ke^{-t/\tau}$. The first term is the **steady state response**, and the second term is the **transient response**. In general, given any sharp change in input, any system will exhibit both forms of response. We are interested in quantifying both. Given the particularly simple form of the Laplace transform of a step function, we often characterize a system by its step response.
- Step inputs are easy to generate. Therefore, if we can model a system as a first-order system, it is possible to numerically obtain the value of K and τ by giving it a step input and plotting its output.

8.4 A Second-Order System

Intuitively, a second-order system arises when the system responds to the input by accumulating and then releasing energy. A classic example is a violin string excited by a bow. As the bow moves over the string, the string distorts, accumulating energy. At some point, the accumulated energy is sufficient to overcome the motion of the bow, and the string returns and then overshoots its neutral position. The process repeats, resulting in an oscillation, powered by the energy of the bow.

Compared to a first-order system, a second-order system is more complex but still relatively easy to analyze. Its transfer function is given by

$$G(s) = \frac{K}{\frac{s^2}{\omega_n^2} + \frac{2\varsigma s}{\omega_n} + 1} \quad \textbf{(EQ 8.9)}$$

A second-order system is characterized by three factors: its **gain** K, its **natural frequency** ω_n (in radians/second), and its **damping ratio** ς (read *zeta*). As before, to understand the physical meanings of these terms, we consider its unit step response. For a step input, we have

$$Y(s) = \frac{K}{s\left(\frac{s^2}{\omega_n^2} + \frac{2\varsigma s}{\omega_n} + 1\right)}$$

We study four cases corresponding to different values of ς.

8.4.1 Case 1 (Undamped System): $\varsigma = 0$

When $\varsigma = 0$,

$$Y(s) = \frac{K}{s\left(\frac{s^2}{\omega_n^2} + 1\right)} = \frac{K\omega_n^2}{s(s^2 + \omega_n^2)} = K\left(\frac{1}{s} - \frac{s}{s^2 + \omega_n^2}\right)$$

Using Table 5.4 in Section 5.8, we compute

$$y(t) = K(1 - \cos\omega_n t) \quad \textbf{(EQ 8.10)}$$

which is shown in Figure 8.4 for $K = 2.5, \omega_n = 2$.

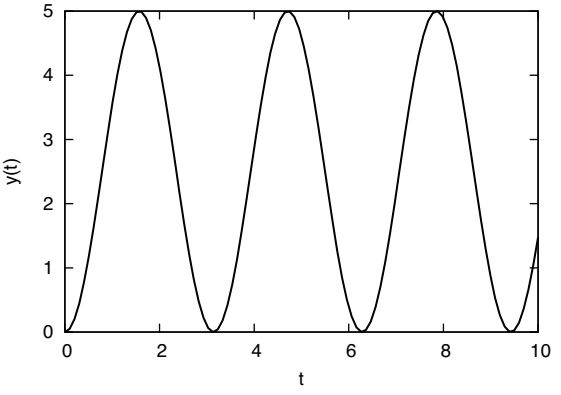


Figure 8.4 Unit step response of an undamped second-order system with $K = 2.5$, $\omega_n = 2$

The output of an undamped second-order system is oscillatory, with a frequency of ω_n. The gain parameter determines the amplitude of the oscillations. These justify naming K and ω_n as the gain and natural frequency, respectively.

8.4.2 Case 2 (Underdamped System): $0 < \varsigma < 1$

When $0 < \varsigma < 1$, the Laplace transform of the step response is given by:

$$Y(s) = \frac{K}{s\left(\frac{s^2}{\omega_n^2} + \frac{2\varsigma s}{\omega_n} + 1\right)} = \frac{K\omega_n^2}{s(s^2 + 2\varsigma s + \omega_n^2)} = K\left(\frac{1}{s} - \frac{(s + \varsigma\omega_n)}{(s + \varsigma\omega_n)^2 + \omega_d^2} - \frac{\varsigma\omega_n}{(s + \varsigma\omega_n)^2 + \omega_d^2}\right)$$

This should be 2 ςωns

where $\omega_d^2 = \omega_n^2(1 - \varsigma^2)$.

Using Table 5.4 in Section 5.8, we compute

$$y(t) = K\left(1 - e^{-\varsigma\omega_n t}\left(\cos\omega_d t + \frac{\varsigma}{\sqrt{1-\varsigma^2}}\sin\omega_d t\right)\right)$$

$$y(t) = K\left(1 - \frac{e^{-\varsigma\omega_n t}}{\sqrt{1-\varsigma^2}}(\sqrt{1-\varsigma^2}\cos\omega_d t + \varsigma\sin\omega_d t)\right)$$

Let $\varsigma = \cos\theta$ so that $\theta = \cos^{-1}\varsigma$. Then $\sqrt{1-\varsigma^2} = \sqrt{1-(\cos\theta)^2} = \sin\theta$, so that

$$y(t) = K\left(1 - \frac{e^{-\varsigma\omega_n t}}{\sqrt{1-\varsigma^2}}(\sin\theta\cos\omega_d t + \cos\theta\sin\omega_d t)\right)$$

From the identity $\sin(A + B) = \sin A \cos B + \cos A \sin B$, we get

$$y(t) = K\left(1 - \frac{e^{-\varsigma\omega_n t}}{\sqrt{1-\varsigma^2}}\sin(\omega_d t + \cos^{-1}\zeta)\right);\ t \geq 0 \qquad \textbf{(EQ 8.11)}$$

This should be cos -1 ς

From the form of Equation 8.11, we see that the sinusoidal oscillation is modulated by a decaying exponential of the form $e^{-\varsigma\omega_n t}$. For a fixed value of ω_n, as ς increases, the exponent dies down more rapidly, damping the system. This justifies calling ς the damping ratio. We also see that the phase of the sinusoid is shifted (compared to the undamped system) by a phase angle $arc\tan\frac{\sqrt{1-\varsigma^2}}{\varsigma}$, a function of ς.

Figure 8.5 shows $y(t)$ for different values of ς, while keeping the values of K and ω_n the same as in the case of the undamped system: $K = 2.5, \omega_n = 2$. As ς approaches 1, the transient response of system becomes less oscillatory. For example, when $\varsigma = 0.2$, the system continues to oscillate even after five time periods (i.e., t=10). On the other hand, when $\varsigma > 0.8$, the output steadily approaches the asymptotic value of 2.5.

Intuitively, the damping ratio determines the responsiveness of the system. When this ratio is small, the system is "jumpy," responding immediately to a stimulus but taking a long time to reach steady state after receiving a shock. As the ratio increases, the system is more sluggish but reaches its asymptote smoothly.

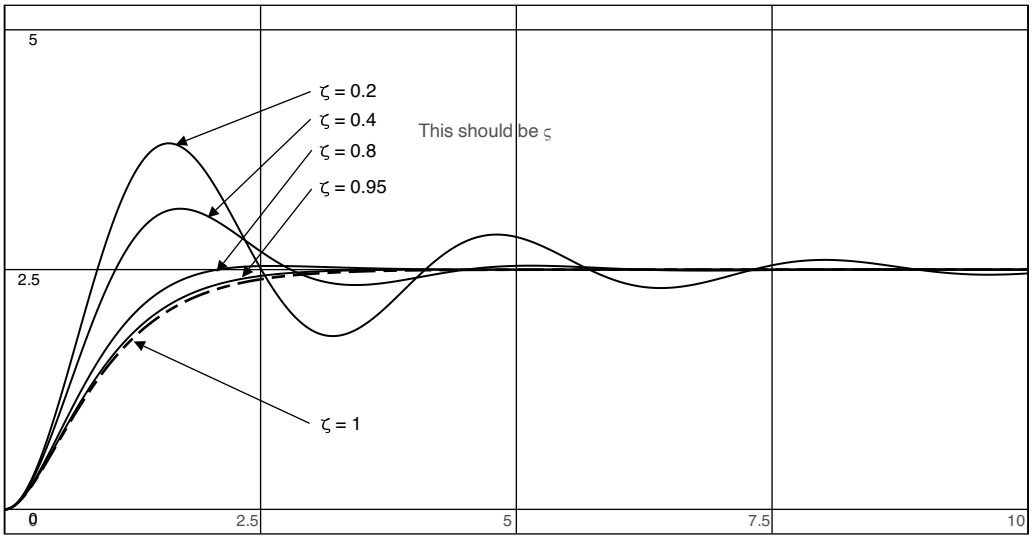


Figure 8.5 Unit step response of an underdamped second-order system with $K = 2.5, \omega_n = 2; \varsigma = 0.2, 0.4, 0.8, 0.95, 1.0$

8.4.3 Case 3 (Critically Damped System): ς = 1

When $\varsigma = 1$, we say that the system is **critically damped**. When subjected to a unit step input, this system is neither too responsive nor too sluggish, rising smoothly to its asymptotic value. Of course, choosing parameters to achieve critical damping for any real system can be a challenge!

To mathematically study such a system, we use the partial fraction expansion of $Y(s)$ as before to find

In this formula, replace ω with ω_n

$$Y(s) = \frac{K\omega^2}{s(s^2 + 2\omega s + \omega^2)} = K\left[\frac{1}{s} - \frac{\omega}{(s+\omega)^2} - \frac{1}{s+\omega}\right]$$

$$y(t) = K[1 - e^{-\omega_n t} - \omega_n t e^{-\omega_n t}] \quad \textbf{(EQ 8.12)}$$

Note that the output $y(t)$ has no sinusoidal components. The corresponding function is shown in Figure 8.5. Comparing the output of a critically damped system with that of an undamped and underdamped system, it should be clear why a critically damped system is ideal.

8:01

8.4.4 Case 4 (Overdamped System): ς > 1

When the damping ratio exceeds 1, the output of a second-order system becomes less and less responsive to a step input. Such a system is **overdamped**. We can mathematically study the system as follows. We have

$$Y(s) = \frac{K\omega_n^2}{s(s^2 + 2\varsigma\omega_n s + \omega_n^2)}$$

The roots of the second term in the denominator, from the quadratic formula, are

$$\frac{-2\varsigma\omega_n \pm \sqrt{4\omega_n^2\varsigma^2 - 4\omega_n^2}}{2},$$

so that

$$Y(s) = \frac{K\omega_n^2}{s(s + \varsigma\omega_n + \omega_n\sqrt{\varsigma^2 - 1})(s + \varsigma\omega_n - \omega_n\sqrt{\varsigma^2 - 1})}$$

For convenience, let $\gamma = \sqrt{\varsigma^2 - 1}$. From the inverse Laplace transform of the partial fraction expansion, we find that

$$y(t) = K\left[1 + \frac{e^{-(\varsigma+\gamma)\omega_n t}}{2\gamma(\varsigma+\gamma)} - \frac{e^{-(\varsigma-\gamma)\omega_n t}}{2\gamma(\varsigma-\gamma)}\right] \quad \textbf{(EQ 8.13)}$$

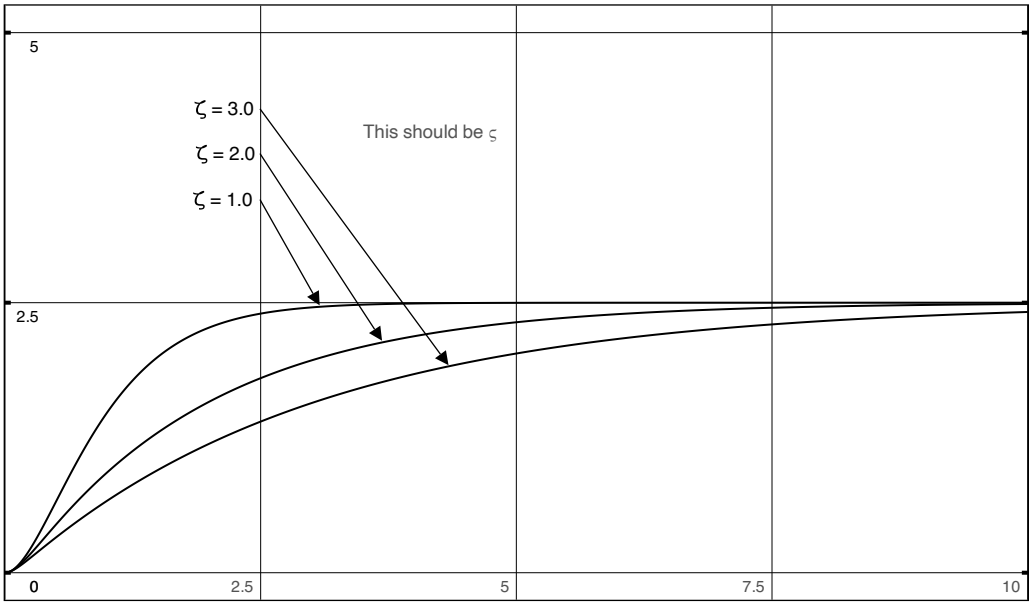


Figure 8.6 Unit step response for an overdamped system with $K = 2.5$, $\omega_n = 2$, $\varsigma = 1.0, 2.0, 3.0$

The step response has no sinusoidal components, being the difference between two exponential curves. The function is plotted in Figure 8.6, with the same values of K and ω_n as before. We see that as the damping ratio increases, the system takes progressively longer to reach its asymptote.

To summarize, the behavior of a second-order system is controlled by its gain, damping ratio, and natural frequency parameters. The gain controls the steady state response to a unit step input. When underdamped, the natural frequency controls the step-response oscillation frequency. Finally, the damping ratio controls the responsiveness of the step response. In designing a control system, it is usually possible to choose arbitrary values for these three parameters. Our goal, therefore, is to tune the system to achieve the desired gain and critical damping.

In practice, no system is truly a second-order system: This is just a convenient approximation. However, the system gives us valuable insights into the behavior of many real systems, as the next example demonstrates.

EXAMPLE 8.5: ROUTE FLAPPING AS A SECOND-ORDER SYSTEM

In a computer network, the dynamic selection of least-delay routes between two nodes can be modeled as a second-order system. Consider two nodes A and B that are connected by two edge-disjoint paths. Suppose that all the traffic

from A to B goes over the first path. This load increases the delay on this path, which we can model as the accumulation of energy. At some point, the energy on the path is so large that the routing algorithm running at node A decides to route traffic destined to B on the second path. This reduces the energy on the first path and increases it on the second. As before, this results in an over-accumulation of energy on the second path, resulting in an oscillatory system exhibiting **route flapping**.

To make this concept more precise, consider a network in which both paths from A and B have a capacity of 1 unit/second. As long as the load from A to B is smaller than 1 unit/second, both paths are uncongested, and the system does not exhibit oscillations. However, suppose that A injects load at the rate of 2 units/second into the first path. This will result in the buffer on the path building up at the rate of 1 unit/second. Suppose that the routing protocol running at A switches paths when the path delay exceeds 5 seconds. This will happen at $t = 5$ seconds. At this point, the load switches to the second path, and the queue there will build up at the rate of 1 unit/second until at time $t = 10$ seconds, traffic will revert to the first path. Clearly, the oscillations have a period of 10 seconds.

EXAMPLE 8.6: DAMPING ROUTE FLAPPING

We saw the oscillatory behavior of a naive threshold-based routing protocol in Example 8.5. We can use the insights from second-order systems to reduce oscillations by using damping. The essence of damping is to reduce system responsiveness. We use this insight as follows: Suppose that the routing algorithm, instead of sending *all* data on the shortest path, distributed load inversely proportional to the measured path delay. Then, the load from A to B would be spread on both paths, increasing overall capacity. Moreover, if there were a sudden increase in delay along one path, load would be proportionally redistributed to another path. By reacting less aggressively to an increase in load, this damped routing algorithm would reduce the oscillatory behavior of the network. The open shortest path first (OSPF) routing protocol in the Internet allows multipath routing for equal-cost paths, so this form of damping is feasible even in practice.

8.5 Basics of Feedback Control


12:58

We are now in a position, finally, to investigate the basics of feedback control. We use the transfer function model to study a simple feedback system. We also investigate the goals of a control system and learn how these goals can be achieved using feedback control.

Recall the abstract model of a continuous, linear, time-invariant control system in Figure 8.1. Assume that all inputs and outputs are scalar. Then, we can use transfer functions to model the behavior of each of the control blocks. That is, if the input to a control block G is the signal $u(t)$ and its output is the signal $y(t)$, we can model the control block by using the transfer function $G(s)$, so that the Laplace transform of the output, $Y(s)$, is merely the product of the transfer function and the Laplace transfer of the input $U(s)$:

$$Y(s) = G(s)U(s)$$

This allows us to redraw Figure 8.1, replacing each control block by its corresponding transfer function and each signal by its Laplace transform, as shown in Figure 8.7.

The use of the transfer function model allows us to state the system equations as follows (we omit the argument s for clarity):

$$Y = GU + GW \quad \textbf{(EQ 8.14)}$$

$$E = R - B = R - HY \quad \textbf{(EQ 8.15)}$$

$$U = DE = D(R - HY) \quad \textbf{(EQ 8.16)}$$

From Equations 8.14, 8.15, and 8.16, we can write

$$Y = GDR - GDHY + GW$$

$$Y(1 + GDH) = GDR + GW$$

$$Y = \left(\frac{GD}{1 + GDH}\right)R + \left(\frac{G}{1 + GDH}\right)W \quad \textbf{(EQ 8.17)}$$

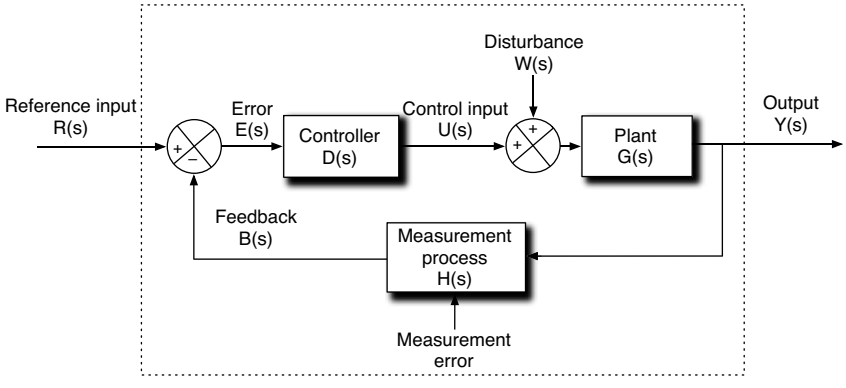
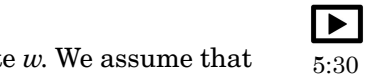


Figure 8.7 Transfer function model of a single-input single-output linear time-invariant feedback control system

Equation 8.17 is the fundamental equation of a feedback control system. It shows that the output is a linear combination of two terms, one arising from the Laplace transform of the reference signal $R(s)$ and the other from the Laplace transform of the disturbance $W(s)$. Note that the only variable that can be modified by a system designer is D, which represents the controller. A common situation is that the measurement process simply feeds the output back to the input, so that $H(s) = 1$: Such a system is called a **unity feedback system**.

We now use this equation to investigate the goals of a control system and how they are achieved by a feedback control system. Essentially, we will compare the controlled system whose output is given by Equation 8.17 with a system with no feedback (i.e., $H = 0$), whose output is given by $Y = G(DR+W)$.

8.5.1 System Goal


5:30

The primary goal of any controller is to ensure that $y = r$ despite w. We assume that we can design the controller, and therefore choose D, more or less as we wish, within some limits, to meet this goal.

It is illuminating to consider how this system goal is achieved in the context of three systems: an uncontrolled system, a controlled system with no feedback, and a system with feedback control. The output of an uncontrolled system is affected by disturbances, and so the control goal is achieved only if $w = 0$. In contrast, a controller that lacks feedback can compensate for a disturbance by predicting it. If it can predict disturbances accurately, it can achieve the system goal. For instance, suppose that an oven temperature controller knows that every time the oven door is opened, a certain amount of heat is lost, and so the heating element needs to be turned on for a certain duration. This controller could maintain oven temperature despite a disturbance created by a door opening by sensing the state of the oven door. In contrast, a feedback control system measures the effect of a disturbance on the output (e.g., the effect of a door opening on the oven temperature) and can use the system-transfer function to determine the appropriate input that removes the effect of the disturbance.

In many situations, the command r is held constant for a reasonably long duration of time. In this case, the goal of a controller is to ensure, first, that the output quickly reaches r and second, that the output stays at r despite disturbances. The time taken for the output to get close to r from its initial state is called the **settling time**. The discrepancy between the actual output and the reference input after the settling time is past is called the **steady state error**.

8.5.2 Constraints

3:46

In achieving the system goal, a controller must take the following constraints into account. The constraints are *stability, disturbance rejection, transient response, linearity, measurement error,* and *robustness.* We consider each in turn.

Stability. Feedback control systems can become unstable if they are improperly designed. Stability essentially means that the output returns to the desired value chosen by a command input despite small variations in the control input or the disturbance. Two widely used stability criteria are **bounded-input**, **bounded-output (BIBO) stability** and **zero-input** stability.

- A system is BIBO stable if, given that the system is initially in equilibrium, any input with bounded magnitude results in an output that also is bounded, although the system may potentially oscillate indefinitely within some bound.
- A system is zero-input stable if, given that the input is zero, all initial states result in the system's eventually reaching the zero equilibrium state.

EXAMPLE 8.7: STABILITY

The Web server system of Example 8.2 is BIBO stable if, starting with a zero initial queue size, the request queue size does not exceed some bound for all request arrival streams. The system is zero-input stable if, given that the initial queue length is finite, as long as no more requests are accepted into the queue, the queue size eventually drops to zero.

Feedback control systems typically achieve stability by ensuring that the control does not overreact to a disturbance. We study this in more detail in Section 8.8.

Disturbance Rejection. A control system should reject any changes in the output owing to a disturbance. From Equation 8.17, we see that the noise input W is attenuated by a factor of $\frac{G}{1+GDH}$. In contrast, an uncontrolled system or a system without feedback control is subjected to the entire disturbance. This shows the benefit of feedback in rejecting disturbance.

Transient Response. When a system in its steady state is subjected to a sudden disturbance, it takes a while for the system to return to equilibrium. This is like a jolt to the handlebar of a bicycle: It takes some time for the bicycle to return to its steady path. The behavior of the system during this time is called its **transient**

6:12

response. If a control system is not properly designed, a sharp disturbance can result in the system's becoming unstable or collapsing.

A feedback control system gives designers a control "knob" to shape a system's dynamic response. To see this, we write Equation 8.17 as

$$Y = \frac{GDR + GW}{1 + GDH}$$

Note that the behavior of $y(t)$ is given by the inverse Laplace transform of this expression, which in turn depends on the roots of the denominator. For instance, if all the roots of the denominator have a strictly negative real component, then $y(t)$ is the sum of decaying exponentials so that the transient response eventually dies down. A designer, therefore, can choose D so that the denominator polynomial has this behavior. This intuition lies at the heart of **pole placement**.

Linearity. A control system designed assuming that the system is LTI will not function correctly when the system is driven into a range where it is nonlinear. Therefore, we must be careful to ensure that the system always stays in the linear range and that, if a disturbance pushes it into a nonlinear regime, the control input brings the system back into a linear regime. In a feedback control system, the system is likely to continue to be linear when the feedback error is small. A designer may need to take special measures to deal with the situation when the feedback error is large. For example, in a computer network, when the network is congested, packet losses from overflowing buffers make the system nonlinear. One reasonable control rule is for sources to drastically reduce their data-transmission rates in the presence of packet loss to restore normal operation. (In practice, a TCP source can be viewed as intentionally placing the system into near-congestion to maximize network utilization.)

EXAMPLE 8.8: LINEARITY

Consider the Web server of Example 8.2. This system is linear as long as the request queue is neither overflowing nor underflowing. If the queue is, for example, overflowing, the error term will be large. In this case, a designer could add a special control rule to set the input rate u to 0, so that the queue drains as quickly as possible and the system reenters the linear regime.

Measurement Error. A feedback system measures the output by using a sensing or measurement device. Importantly, the feedback system *cannot* compensate for an error in this sensor. For instance, if a thermostat sensor is off by 1 degree, the controlled system has a steady state error of 1 degree. Therefore, it is critical that the measurement process be as error-free as possible.

A second form of measurement error arises from delays in the measurement process. Consider a thermostat that takes 5 minutes to measure the room temperature. In such a system, once the setpoint is reached, the furnace would be left on 5 minutes too long, raising the room temperature beyond the setpoint. At this point, the furnace would be turned off, and the room temperature would return to the setpoint, then drop below it for at least 5 minutes before corrective action could be taken. It is clear that the delayed measurement results in a persistent oscillation in the output signal (i.e., the room temperature). In general, feedback delays can cause oscillations or even instability. The general approach to deal with such delays is to be cautious in responding to a measurement, recognizing that it may reflect stale input. This can be achieved by damping the control input. We will consider dealing with feedback delays more fully in Section 8.7.

Robustness. A controller is designed subject to many assumptions: that the system is linear, that the system model is accurate, that the measurement is correct, and that the measurement is not subjected to too much delay. Moreover, we must also assume that the system behavior does not change over time, owing to wear and tear or software upgrades. These are strong assumptions. Therefore, it is necessary to design a controller that does not exhibit poor behavior when one or more of these assumptions are violated. The theory of **robust control** deals with the design of control systems that work well despite bounded variations in the system parameters and is described in more advanced textbooks on control theory.

8.6 PID Control



6:04

Proportional-integral-derivative (PID) control, the simplest classical approach to controller design, refers to controllers whose control input is proportional to the error, to the integral of the error, or to the derivative of the error. PID controllers may use two or even three modes simultaneously. We study each type of controller in turn. Note that in all three modes a higher loop gain improves responsiveness.

8.6.1 Proportional-Mode Control

With proportional control, the control signal is a scalar multiple of the feedback error, so that $u = K_p e$ or, in the transform domain, $U = K_p E$, where K_p is called the **loop gain**.[2] The greater the loop gain, the larger the control action corresponding to a disturbance. Therefore, a large loop gain makes the controller more responsive.

2. This is not the same as the system gain. The loop gain is the ratio of the command to the error; the system gain is the ratio of the output to the input, which, in the case of a perfectly controlled system, should be 1.

Note that the control input is proportional to the error, so that, if the system in steady state needs a nonzero control input, there needs to be a nonzero steady state error, also called the **droop**. To keep the steady state error small, we need to make the loop gain large. However, this reduces damping and can make the system unstable. (We will return to this topic when we study system stability in Section 8.8.)

EXAMPLE 8.9: PROPORTIONAL-MODE CONTROL



Consider the design of a proportional controller for the Web server in Example 8.2. Recall that its transfer function $G(s)$ is given by $\frac{1}{s}$. Suppose that the command r is the desired setpoint of the request buffer. Then, the error is given by $r - y$. For proportional control, the control input is proportional to this error.[3] Specifically, the controller chooses an input u such that, if there is no further disturbance, the system output y returns to the desired setpoint r after a time τ. That is,

$$u = \frac{r-y}{\tau},$$

so that the loop gain is $\frac{1}{\tau}$. Recall that $Y = \frac{1}{s}(U - W) = \frac{1}{s}\left(\left(\frac{R}{\tau} - \frac{Y}{\tau}\right) - W\right)$, which can be rearranged to get

$$Y = \frac{R}{1+s\tau} - \frac{W\tau}{1+s\tau}$$

To study the step response of this controller, set $W = 0$ and $R = 1/s$. Then, $Y = \frac{1}{s(1+s\tau)}$. Using partial fractions to expand this,

$$Y = \frac{1}{s} - \frac{1}{s+\frac{1}{\tau}},$$

and taking the inverse Laplace transform, we find $y(t) = 1 - e^{-t/\tau}$. That is, when there is no disturbance, if the input command is a step signal, the output is a step signal (i.e., the desired value because $y = r = 1$) along with a transient response that decays exponentially over time. The smaller the value of τ, the larger the loop gain and the faster the decay of the transient response.

3. If the error is negative (that is, the buffer occupancy is greater than the setpoint), the controller must send "negative" packets into the buffer. For simplicity, we interpret this to mean that the controller sends special "cancel" requests that have the effect of removing pending requests from the Web server's request buffer.

We investigate the steady state error in the step response (assuming zero disturbance) by using the final value theorem, which states that

$$\lim_{t \to \infty} x(t) = \lim_{s \to 0} sX(s).$$

Thus,

$$\lim_{t \to \infty} e(t) = \lim_{s \to 0} sE = \lim_{s \to 0} s\left(\frac{1}{s} - Y\right) = \lim_{s \to 0} s\left(\frac{1}{s} - \frac{1}{s} + \frac{1}{s + \frac{1}{\tau}}\right) = 0$$

This shows that, independent of the loop gain, the steady state error is zero.

8.6.2 Integral-Mode Control


6:34

The motivation for integral control is the intuition that the magnitude of the control input should depend not only on the current value of the error but also the total error built up in the past: The greater this overall accumulated error, the greater the correction. Mathematically, with integral control, $u = K_i \int e$ or $U = K_i \frac{E}{s}$, where K_i is the loop gain. Integral control removes the need for a steady state error term but usually makes the control less stable, as the next example demonstrates.

EXAMPLE 8.10: INTEGRAL-MODE CONTROL

Consider the design of an integral controller for the Web server in Example 8.2 with transfer function $G(s) = \frac{1}{s}$. For integral control, the control input u is proportional to the integral of the error: $e = (r - y)$, so $u = K_i \int (r - y)$ and

$$U = \frac{K_i(R - Y)}{s},$$

where the loop gain is K_i. Now, $Y = \frac{1}{s}(U - W) = \frac{1}{s}\left(\left(\frac{K_i(R - Y)}{s}\right) - W\right)$, which can be rearranged to get

$$Y = \left(\frac{K_i}{s^2 + K_i}\right)R - \left(\frac{s}{s^2 + K_i}\right)W$$

To study the step response of this controller, set $W = 0$ and $R = 1/s$. Then,

1:57

$$Y = \frac{K_i}{s(s^2 + K_i)} = \frac{1}{s} - \frac{s}{s^2 + K_i}.$$

Taking the inverse Laplace transform, we find $y(t) = (1 - \cos\sqrt{K_i}t)$. That is, when there is no disturbance, if the input command is a step signal, the output is the step signal along with a persistent sinusoidal oscillation with magnitude 1 and period $\frac{1}{\sqrt{K_i}}$.

The transient response neither grows nor decays with time: We call a system with such a response **marginally stable**. This demonstrates our claim that the introduction of integral-mode control diminishes system stability. Note that the larger the loop gain, the faster the oscillation frequency.

8.6.3 Derivative-Mode Control

7:37

Derivative-mode control recognizes the fact that control actions (for the same degree of error) should differ depending on whether the error trend is increasing or decreasing. If the error trend (its derivative) is increasing, the controller needs to take stronger corrective action, but when the error trend is decreasing, the controller must back off the control action to prevent overreaction. Thus, derivative-mode control tends to dampen the actions of a controller.

Derivative-mode control cannot be used in isolation, because a constant steady state error, no matter how large, has a zero derivative. To avoid steady state error, derivative-mode control is combined with either proportional-mode or both integral- and proportional-mode control.

Note also that derivative-mode control is very sensitive to high-frequency noise in the measurement process: Spikes in measurement noise are interpreted as a sharp increase in system error. To avoid this problem in practice, measurements must be filtered using a low-pass filter before using them as the basis of derivative-mode control.

Mathematically, with derivative-mode control, $u = K_d\left(\frac{de}{dt}\right)$ or $U = K_d E s$, where K_d is the loop gain. We demonstrate derivative-mode control through an example.

EXAMPLE 8.11: DERIVATIVE-MODE CONTROL

Consider a derivative controller for the Web server in Example 8.2 with transfer function $G(s) = \frac{1}{s}$. For derivative control, the control input is proportional to the derivative of the error, so that $U = K_d(R - Y)s$, where the loop gain is K_d. Now, $Y = \frac{1}{s}(U - W) = \frac{1}{s}((K_d(R - Y)s) - W)$, which can be rearranged to get

$$Y = \left(\frac{K_d}{1 + K_d}\right)R - \left(\frac{1}{s(1 + K_d)}\right)W$$

To study the step response of this controller, we set $W = 0$ and $R = 1/s$. Then, $Y = \frac{K_d}{s(1 + K_d)}$. Taking the inverse Laplace transform, we find $y(t) = \frac{K_d}{1 + K_d}$. That is, when there is no disturbance, if the input command is a step signal, the output is an attenuated step signal with no transient response. Note that the larger the loop gain, the smaller the attenuation in the input signal.

We can investigate the steady state error in the step response (assuming zero disturbance) by using the final-value theorem. We have $Y = \frac{K_d}{s(1 + K_d)}$, so from the final-value theorem, the asymptotic value of $y(t)$ is $\frac{K_d}{(1 + K_d)} \neq 1$, which demonstrates our earlier claim that derivative-mode control results in a nonzero steady state error. However, note that the steady state error is a decreasing function of the loop gain, so the steady-state error can be reduced by increasing the loop gain.

8.6.4 Combining Modes

Each of the three basic control modes—proportional, integral, and derivative—has its advantages and disadvantages. Therefore, it is common to combine modes in a single so-called **PID controller**. Such a controller has three control parameters that correspond to loop gains in each mode. That is, for an error E, we have

$$U = \left(K_p + \frac{K_i}{s} + K_d s\right)E$$
$$D = \left(\frac{K_d s^2 + K_p s + K_i}{s}\right) \quad \text{(EQ 8.18)}$$

The relative contributions of the three modes can be adjusted using the gain factors. Intuitively, the use of proportional mode corrects for error, the use of integral mode removes steady state error, and the use of derivative mode corrects for instability that may be produced by the integral mode and dampens the control action.

We will not be covering Section 8.7, but I've left it in, in case anyone is interested.

8.7 Advanced Control Concepts

Having studied the basic PID control, we now examine, at an intuitive level, some more advanced control concepts that are applicable to control systems in computer networks.

8.7.1 Cascade Control

When trying to accomplish a complex control objective for a plant or process that exhibits multiple timescales of behavior, it is often useful to layer control. For example, with two layers of control, a lower-layer controller, operating at a faster timescale, controls a portion of the system to achieve a desired setpoint, and the higher-layer controller, operating at a slower timescale, dynamically chooses the setpoint to achieve the top-level control objective. This is called **cascade control**.

EXAMPLE 8.12: CASCADE CONTROL

Consider the use of cascade control to optimize the communication between a Web browser client and a Web server. A Web browser can have multiple connections open to multiple Web servers, where each connection queries for and fetches a set of data objects to be rendered on the screen.[4] Ideally, the connections should make progress at rates proportional to the amount of data being transferred over the connection, so that all the elements of the Web page can be rendered at the same time.

Recall that query-request rate control operates at the tens to hundreds of milliseconds timescale and adjusts the sending rate from a source to ensure that the buffer-occupancy level at the Web server is close to a desired setpoint. Recall from Example 8.2 that the greater this setpoint, the greater the achievable end-to-end throughput. This suggests the use of a higher-layer controller that dynamically modifies the setpoint of the buffer-occupancy level to achieve the overall control objective. Specifically, the higher-layer controller has, as its

4. As a simplification, we assume that all queries are of equal size and take equal amounts of time to process, so that we need to control only the number of queries, independent of their size. Generalizing from this simplified case to the true situation is left as an exercise to the reader.

control goal, the desire to have all connections make equal fractional progress on their queries. The error term for this controller is the difference between the actual fractional progress made by a connection and the mean fractional progress made by all the clients. If this error term is negative, the connection is a lagging connection, and its setpoint is increased using proportional-, integral-, or differential-mode control. On the other hand, if this error term is positive, the connection is a leading connection, and its setpoint is similarly decreased.

To make this example concrete, consider a browser that has three connections to three different Web servers, where the first connection fetches three query responses, the second fetches five query responses, and the third fetches ten responses. Therefore, they should make progress at the relative rates of 3:5:10. Suppose that at some point in time, the higher-layer controller finds that the first connection has retrieved two responses, the second has retrieved three responses, and the third has retrieved five responses. The fractional progress made, therefore, is 2/3:3/5:5/10 = 0.66 : 0.6 : 0.5. The mean progress is 0.59. Therefore, the higher-layer controller decreases the setpoint of the first connection, leaves the second connection's setpoint nearly unchanged, and increases the setpoint of the third connection.

8.7.2 Control Delay

Our discussion so far has assumed that the measurement of the plant output is instantaneously available to the controller. This is, of course, almost never the case. Delays in measurement are particularly challenging for controllers developed for computer networks, where measurement delays can be significant. We first describe the mathematical modeling of measurement delay and then describe the effects of delay on control.

Measurement delays are easy to represent mathematically. Recall from Table 5.3 in Section 5.8 that a delay of τ seconds corresponds to multiplying the Laplace transform of a signal by $e^{-s\tau}$. Therefore, with feedback delay, we find that the error E is given by

$$E = R - e^{-s\tau}Y \qquad \textbf{(EQ 8.19)}$$

We can then use this input as the basis for, say, proportional control, where $U = K_pE = K_p(R - e^{-s\tau}Y)$. The subsequent analysis of this system proceeds as before, except that going from the Laplace domain to the time domain is complicated because we cannot use partial fraction expansion, which applies only to ratios of polynomials. Instead, we can use the **Pade approximations** of the exponential function:

$$e^{-s\tau} \approx \frac{1 + \frac{(-s\tau)}{2}}{1 - \frac{(-s\tau)}{2}}$$

$$\approx \frac{1 + \frac{(-s\tau)}{2} + \frac{(-s\tau)^2}{12}}{1 - \frac{(-s\tau)}{2} + \frac{(-s\tau)^2}{12}}$$

which replace the exponential with increasingly accurate polynomial ratios. More elaborate Pade approximations can be found in the literature.

The general treatment of control systems with delays is complex and beyond the scope of this book. However, two observations are relevant. First, a practical approach to deal with delays is to predict the current system state from past measurements, using an appropriate predictor. Once this is done, the problem reduces to the standard zero-delay control system. Many sophisticated techniques for time-series prediction can be used for this purpose. Second, the general effect of delays is that the controller does not know the response to its prior control action. To prevent oscillations and instability, the controller should choose to be cautious in its actions, delaying its next control action until it has learned the response to its prior action. This leads to a fundamental trade-off between responsiveness and stability. In most practical systems, stability is more important and comes at the cost of decreased responsiveness. For example, in TCP, to dampen the control input, control actions are taken no faster than once per round trip time, and the response to a loss event is artificially delayed by up to half a second. This reduces oscillations, though it also reduces responsiveness.

EXAMPLE 8.13: EFFECT OF CONTROL DELAY

Consider a transport-layer flow-control algorithm that takes control actions in one of two ways (Figure 8.8). If it receives an acknowledgment with the explicit forward congestion notification (EFCN) bit set to zero, indicating that the corresponding data packet did not experience congestion, it increases its sending rate additively; otherwise, it decreases its sending rate multiplicatively. This is a controller with an undamped response. In contrast, consider a flow-control algorithm that collects the EFCN bits set on packets sent during each window of duration one round trip time (measured from the start of the connection), then takes control actions once per round trip time, depending on whether the majority of the bits were set. This controller dampens its response.

Let us do a thought experiment to study the behavior of the two systems. To exaggerate the difference in their behavior, we will assume that the source has

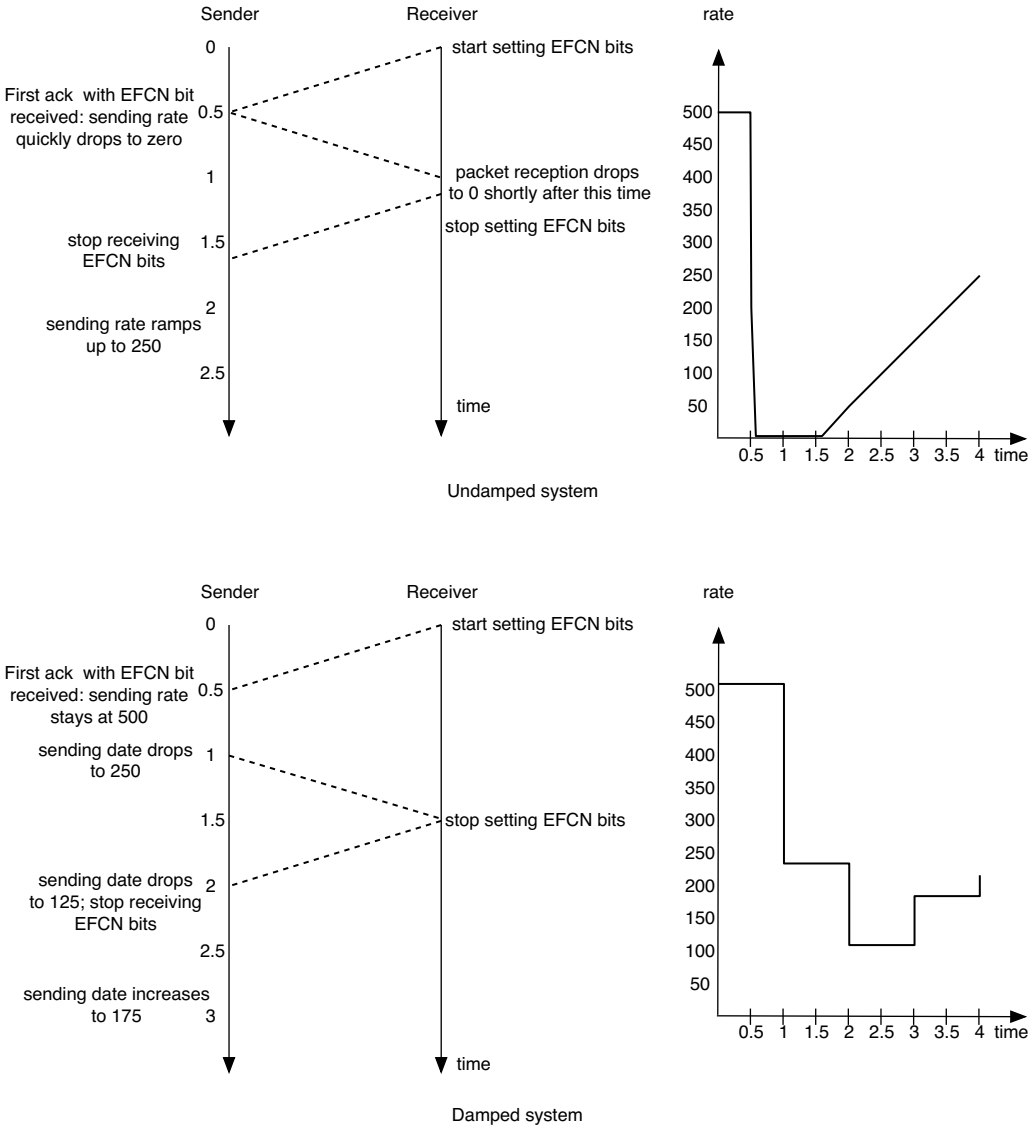


Figure 8.8 Figure for Example 8.13

an infinite number of packets to send and that its fastest sending rate is 1,000 packets per second. Assume also that the delay from the time that an EFCN bit is set to the time that it is received by a sender is 0.5 seconds and that the end-to-end delay is also 0.5 seconds (i.e., the single bottleneck link is next to the receiver), so that the round trip time (RTT) delay is 1 second. Finally, assume that the capacity of the bottleneck link is 500 packets per second, the

additive increase factor is 50 packets/RTT, and that the multiplicative decrease factor is 0.5.

We conveniently define the system to be in equilibrium when the source is sending packets at 500 packets/second and the bottleneck link is serving packets at 500 packets/second. Consider the time evolution of control for the damped and undamped controllers when the system is initially at equilibrium and when, at time 0, the bottleneck service rate decreases abruptly to 250 packets/second, owing to the arrival of another flow.

With both the undamped and the damped controllers, packets with EFCN bits arrive to the sender at time 0.5. With the undamped system, on the arrival of the first acknowledgment packet with an EFCN bit set, the source immediately reduces its sending rate to 250 packets/second, further reducing its rate with every additional acknowledgment. Note that in equilibrium, the source has 500 packets outstanding, all with bits set. So the source reduces its rate to nearly 0 by time 1.5, when the last of the EFCN bits is received. Then, no more bits are set, so the source increases its rate by 50 packets/RTT, reaching its new equilibrium value of 250 at time 6.5. The subsequent time evolution is straightforward. Note that the effect of this immediate response is to quickly drain the bottleneck queue of any packets that are buffered there owing to the drop in the link-service rate.

With the damped system, the source continues to send at the rate of 500 packets/second until time 1 because it takes control actions only at times 1, 2, 3, and so on. At time 1, it has received 500 acknowledgments, of which 250 have bits set and 250 do not. Assume that, in this borderline case, the source reduces its rate to 250 packets/second at time 1. In the time interval from 1 to 2 seconds, it sends 250 packets but receives 500 packets with EFCN bits set (left over from the first round trip time). So, it reduces its rate once more to 125 packets/second. No more bits are set subsequently, and the source ramps up its rate to 250 over the next 3 seconds. Note that, owing to the delayed response, the bottleneck queue is drained much more slowly than with the undamped system.

Damping decreases responsiveness, so the effect of a disturbance persists longer, but the magnitude of the impulse response diminishes, and the system returns to equilibrium sooner. This is characteristic of any control system with control delays.

8.8 Stability

Consider a system that is not subject to a disturbance for some length of time. We expect such a system to eventually settle down so that its output y matches the

command input r. A system whose output persists at its setpoint is said to be in **equilibrium**, or **relaxed**. By convention, we denote this state—and the corresponding command and output—by a **zero** vector. In many cases, this ideal equilibrium state may never be achieved. Yet it is important because it is the state that a system *strives* to achieve.

Suppose that a system in equilibrium is subjected to a time-limited disturbance. Ideally, if the disturbance is small, the change in the output also should be small. Moreover, the system should rapidly return to equilibrium when the disturbance ceases. We call such an ideal system a **stable** system.

To further grasp the concept of stability, imagine a cone that is lying on its side and another that is balanced on its point. A small disturbance to a cone balanced on its point results in a large change (it falls down); when the disturbance is removed, the cone does not return to its previous balance. In contrast, a small disturbance to a cone lying on its side results in a small change in position, although it may not return to the original equilibrium state when the disturbance is removed.

Besides these two responses to a small disturbance, a third possible response is for the system to oscillate, where the magnitude of the oscillations decays over time. This is akin to a rocking chair or a guitar string subjected to a small impulse disturbance. Depending on the desired output behavior of the system, such a response could be characterized as either stable or unstable.

When designing a controller, it is very desirable that the controlled system be stable. In this section, we study the conditions under which a controlled system can be guaranteed to be stable. Before doing so, it is worth distinguishing between two types of stable responses.

1. A system is said to be **bounded-input**, **bounded-output** (**BIBO**) stable if the output of the system is bounded in magnitude by some value M as long as the disturbance is bounded in magnitude by m.
2. A system is said to be **zero-input** stable if, for all initial states of the system, including nonequilibrium states, when both the command and the disturbance inputs are zero, the system eventually reaches the equilibrium state.

Both BIBO and zero-input stability are relatively easy to characterize for LTI systems. However, for time-varying or nonlinear systems, a system may be stable for some classes of disturbances but not others, or may be stable for small disturbances but not large ones. Such systems may also have multiple equilibrium states and may move from one equilibrium state to another when subject to small disturbance, much like a round pencil lying on its side when subjected to a small torque. Such multiple equilibria arise naturally in computer networks.

EXAMPLE 8.14: METASTABILITY IN THE TELEPHONE NETWORK

The telephone network core is organized as a clique where traffic from any node to any other node follows a one-hop path. This routing is simple but inefficient, in that the one-hop path does not make use of capacity available on two-hop paths. Consider a clique where all links have unit capacity so that the one-hop path from any node A to any other node B has capacity 1. If the clique has n nodes, there are an additional $n - 2$ two-hop paths between them also with unit capacity. So node A could send traffic to node B at a rate as high as $n - 1$. However, using two-hop paths comes at the cost of using twice the network capacity per traffic flow (i.e., two links instead of one). This has the following effect: If the network has a high traffic load so that most one-hop paths are nearly at capacity, the addition of a small additional load on a one-hop path could cause diversion of the excess load to one of the alternative two-hop paths. If these two-hop paths also are close to capacity, additional traffic on either of these hops would cause the use of additional two-hop paths. At high loads, this process continues until most traffic traverses two-hop paths, reducing network efficiency to 50%. This inefficient state turns out to be persistent: Even if traffic volume decreases, most traffic would still traverse two-hop paths. Thus, the system has more than one equilibrium.[5]

Despite the presence of multiple equilibria, note that the system is both BIBO and zero-input stable. To see this, we define the input to the system to be its traffic load and the output as the carried load. The system is in its relaxed state when traffic load is zero, corresponding to a zero carried load. If a small disturbance adds traffic to the system, it is carried on a one-hop path, and the carried load equals the traffic load. Therefore, as long as the input is bounded, so too is the output, showing BIBO stability.

To prove zero-input stability, consider a network in which the traffic load is nonzero at time zero but there is no additional load after time zero. In this case, once the traffic is delivered to its destination, the network always eventually returns to the zero state, as desired.

The instability, or rather, change in state from one equilibrium state to another, is caused by a disturbance when the system load is *high*. We see, therefore, that BIBO and zero-input stability criteria are rather weak. Perhaps a better way to view them is that a system that fails even these two weak stability criteria is not particularly robust.

5. This unstable equilibrium can be avoided by reserving a fraction of each link's capacity for one-hop paths.

EXAMPLE 8.15: COMPUTER NETWORK STABILITY

A proof along the lines of Example 8.14 shows that nearly all computer networks are trivially both BIBO and zero-input stable if we accept the definition that the output of the system is the overall carried load and the zero state is the zero-load state. We could also define the output of the network as the sum of all the router buffer occupancies, so that a congested network would have a large output amplitude. But all buffers, in practice, are bounded in size. So, no matter what the input load, they cannot exceed a predefined constant. From this perspective, again, computer networks are trivially stable. (The most common manifestation of marginal stability is probably the presence of a persistent oscillation, as we saw in Example 8.5.)


7:07

8.8.1 BIBO Stability Analysis of a Linear Time-Invariant System

This section presents the conditions under which a continuous single-input, single-output LTI system is BIBO stable. If the system has input $r(t)$, output $y(t)$, and impulse response $g(t)$, then, from Equation 5.28 in Section 5.4:

$$y(t) = \int_{-\infty}^{\infty} g(\tau)r(t-\tau)d\tau = \int_{0}^{\infty} g(\tau)r(t-\tau)d\tau$$

where the second step follows from the fact that for a causal system, the impulse response is zero when $t < 0$. The magnitude of the output is bounded by

$$\left|\int_{0}^{\infty} g(\tau)r(t-\tau)d\tau\right| \leq \int_{0}^{\infty} |g(\tau)||r(t-\tau)|d\tau \leq r_{max}\int_{0}^{\infty} |g(\tau)|d\tau$$

where r_{max} is the maximum value of the input. The output is bounded as long as the quantity

$$\int_{0}^{\infty} |g(\tau)|d\tau,$$

which is the integral of the absolute value of the impulse response $g(t)$, is bounded. To compute this quantity, we first consider the Laplace transform of the impulse response: the transfer function $G(s)$. It can be shown that the transfer function of any LTI system that can be modeled as a set of differential equations can be written as the ratio of two polynomials in s:

$$G(s) = \frac{b_0 s^m + b_1 s^{m-1} + \ldots + b_m}{s^n + a_1 s^{n-1+\ldots+a_n}} \quad \textbf{(EQ 8.20)}$$

The impulse response $g(t)$ of any real system is real, that is, it has no complex coefficients. Therefore, by the definition of the Laplace transform, its Laplace transform $G(s)$ also has real coefficients, and the roots of its denominator polynomial $s^n + a_1 s^{n-1+\ldots+a_n}$ must be either real and distinct, real and repeated, or conjugate complex; and it cannot have nonconjugate complex roots. Using partial fraction expansion (see Section 8.11), such a transfer function can be written in the form

$$G(s) = \left\{ \begin{array}{c} \dfrac{a_1}{(s-\alpha_1)} + \dfrac{a_2}{(s-\alpha_2)} + \ldots + \text{other distinct real roots} \\ \dfrac{a_k + jb_k}{(s-(\alpha_k + j\beta_k))} + \dfrac{a_k - jb_k}{(s-(\alpha_k - j\beta_k))} + \ldots + \text{other complex conjugate roots} \\ \dfrac{a_{m(r)}}{(s-\alpha_m)^r} + \dfrac{a_{m(r-1)}}{(s-\alpha_m)^{r-1}} + \ldots + \dfrac{a_{m(1)}}{(s-\alpha_m)} + \text{other repeated roots} \end{array} \right\} \quad \textbf{(EQ 8.21)}$$

What does this look like in the time domain? The Laplace transform is linear, so we can take the inverse transform term by term.

- For a term of the form

$$\frac{a_i}{(s-\alpha_i)},$$

the corresponding time-domain term is $a_i e^{\alpha_i t}$, which decays exponentially to zero whenever $\alpha_i < 0$; that is, the root lies on the *X*-axis in the left half of the complex *s* plane. In this case, the integral of the absolute value of the impulse response is bounded, and the system is stable. If $\alpha_i = 0$, the output is constant, the integral is unbounded, and the system in unstable.

- For complex conjugate terms of the form



5:04

$$\frac{a_k + jb_k}{(s-(\alpha_k + j\beta_k))} + \frac{a_k - jb_k}{(s-(\alpha_k - j\beta_k))},$$

the corresponding time-domain term is $2e^{\alpha_k t}(a_k \cos\beta t - b_k \sin\beta t)$. Note that the expression in the parentheses is the difference between two sinusoids and therefore is bounded in magnitude. The output corresponding to this pair of terms decays to zero, corresponding to a bounded integral, if and only if α_k, the real part of the complex conjugate root, is negative, so that the pair of roots lie in the left half of the complex *s* plane. If the real part is zero (i.e., the

root lies on the j axis), the system oscillates, and this oscillation does not decay over time. This system is marginally stable.

- A similar but somewhat more complicated analysis shows that if the system has repeated real or complex roots, then the corresponding time-domain output decays to zero, and the integral of the impulse response is bounded if and only if the real part of the root lies strictly in the left half of the complex s plane: Roots on the j axis lead to marginally stable systems, and repeated roots on the j axis or roots in the right half of the complex s plane lead to unstable systems.

Summarizing, we see that an LTI system is BIBO stable if and only if *all* the roots of the denominator polynomial of its transfer function lie in the left half of the complex s plane. If even one root lies on the right half of the plane, the system is unstable. For the special case of conjugate roots on the j axis: (1) if the pairs of conjugate roots on this axis are distinct, the system is oscillatory but bounded; (2) if the pairs of conjugate roots on this axis are repeated, the system is unstable. This simple characterization allows us to quickly determine whether a single-input, single-output LTI system is BIBO stable simply by finding the roots of its denominator polynomial: These are also called the **poles** of the system. This can be determined using numerical algorithms, such as the Jenkins-Traub algorithm.

EXAMPLE 8.16: BIBO STABILITY OF AN LTI SYSTEM

Determine the stability of an LTI system with denominator polynomial $3s^4 + 5s^3 + 7s^2 + 12s + 1$.

Solution:

Using a numerical solver, we find that the roots of this polynomial are –0.08753, –1.6486, $0.03475 \pm j1.51936$. Since the conjugate roots lie in the right-half plane, the system is unstable.

EXAMPLE 8.17: BIBO STABILITY OF AN LTI SYSTEM

Determine the stability of an LTI system with denominator polynomial $s^4 + 50s^2 + 625$.

Solution:

We factor the polynomial as $(s^2 + 25)^2 = ((s + j5)(s - j5))^2$, so that there are two repeated roots at $\pm j5$. The system has repeated roots on the j axis and is therefore unstable.

EXAMPLE 8.18: STABILITY OF A WEB SERVER CONTROLLER

Determine the stability of the proportional-, integral-, and derivative-mode controllers of Examples 8.9–8.11.

Solution:

For the proportional-mode controller, we have $Y = \frac{R}{1+s\tau} - \frac{W}{s(1+s\tau)}$. Ignoring the disturbance, the transfer function of the controlled system is given by $G = \frac{1}{1+s\tau}$. The root is at $s = -\frac{1}{\tau}$. Since $\tau > 0$ (it is a time constant), the system's single pole, which corresponds to this root, is always in the left half of the complex s plane and is therefore always BIBO stable.

For the integral-mode controller, we have $Y = \left(\frac{K_i}{s^2+K_i}\right)R$ so that $G = \left(\frac{K_i}{s^2+K_i}\right)$ and the roots are at $\pm jK_i^{1/2}$. The conjugate poles on the j axis indicate that the system is oscillatory and therefore bounded but BIBO marginally stable.

For the derivative mode-controller, we have $Y = \left(\frac{K_d}{1+K_d}\right)R$, so $G = \left(\frac{K_d}{1+K_d}\right)$, which is a constant that is never 0 (i.e., the system has no pole). The effect of the controller is to act as a constant scalar multiplier to the Laplace transform of the control input. If the input is bounded, so too is the output, which implies that the system is BIBO stable.

See the video link above

There is one corner case that requires special care. Suppose that $G(s)$ has a single set of complex conjugate roots on the j axis, which corresponds to a marginally stable bounded oscillatory system. Suppose that it is excited by an input $R(s)$ that also has complex conjugate roots at the same locations on the j axis. Then, $Y(s) = G(s)R(s)$ has conjugate roots of multiplicity 2 on the j axis, making it both unbounded and unstable. The physical interpretation of this phenomenon is that an input of a sinusoid at a frequency that exactly matches one of the natural frequencies of the system leads to instability owing to **resonance**.

8.8.2 Zero-Input Stability Analysis of a SISO Linear Time-Invariant System



8:39

We now turn our attention to zero-input stability of a single-input, single-output (SISO) system. In this context, zero input means that the command given to the

system (i.e., r) is 0 and does not change over time. Recall that a system is zero-input stable if, in the absence of any input or disturbance, no matter what the initial state at time zero, the system always eventually returns to its equilibrium state. It is convenient to study zero-input stability by using the state space representation of a system (Equation 8.2):

$$\begin{aligned} \dot{\boldsymbol{x}} &= A\boldsymbol{x} + \boldsymbol{b}u + \boldsymbol{F}\boldsymbol{w} \\ y &= \boldsymbol{c}\boldsymbol{x} + du + \boldsymbol{e}\boldsymbol{w} \end{aligned}$$

We have $\boldsymbol{w} = u = 0$, so the state evolution is given by

$$\dot{\boldsymbol{x}} = A\boldsymbol{x}$$

Taking the Laplace transform of both sides,

$$s\boldsymbol{X}(s) - \boldsymbol{x}(0) = \boldsymbol{A}\boldsymbol{X}(s)$$

$$\begin{aligned} \boldsymbol{X}(s) &= (s\boldsymbol{I} - \boldsymbol{A})^{-1}\boldsymbol{x}(0) \\ &= \frac{[C(s\boldsymbol{I} - \boldsymbol{A})]^T}{|s\boldsymbol{I} - \boldsymbol{A}|}\boldsymbol{x}(0) \end{aligned} \qquad \textbf{(EQ 8.22)}$$

where the second step expands the inverse in terms of the cofactor and the determinant (see Section 3.4.4). The numerator is a matrix whose elements are polynomials in s, and the denominator is polynomial in s. Therefore, $\boldsymbol{X}(s)$ is a vector whose elements are ratios of two polynomials in s. As before, we can expand each element as a partial fraction. For the system to be zero-input stable, we require that every element of the state vector eventually decay to zero. This will be true if and only if the roots of the equation $|s\boldsymbol{I} - \boldsymbol{A}| = 0$, which are the eigenvalues of $\boldsymbol{A}$, lie in the left half of the complex plane. Because of the importance of this equation, it is also called the **characteristic equation** of the system. Under mild conditions, it can be shown that the roots of the characteristic polynomial are, in fact, identical to the poles of the corresponding transfer function. Therefore, under these conditions, a system is either both BIBO and zero-input stable or neither, and only one of the tests needs to be carried out.

8.8.3 Placing System Roots

The roots of the denominator of the system-transfer function or of the characteristic equation are critical in determining its stability. If a system designer has one or more free parameters, or "control knobs" to control system behavior, these must be chosen such that these roots for the controlled system lie in the left half of the complex plane. This is called **pole placement**, since each root corresponds to a system pole.

Before the advent of digital computers, finding the roots of the characteristic equation was nearly impossible other than for trivial systems. System engineers, therefore, came up with approximation techniques, such as the root-locus method, Bode plots, and Nyquist plots to roughly determine root locations as a function of the system parameters. Today, sophisticated numerical algorithms have simplified the process of finding the roots of any polynomial. Nevertheless, an appreciation of the influence of the system poles on its behavior is critical in designing a robust and stable control system. We will not explore this complex topic in any detail. However, it should be clear that the farther to the left of the j axis that we place a pole, the faster the decay of impulse response transients, which makes the system more responsive. However, this can also lead to instability owing to overcorrection. The goal of the system designer is to balance responsiveness with stability.

8.8.4 Lyapunov Stability


16:07

Most real systems, such as computer networks, are neither linear nor time invariant. Yet we would like to understand whether they are stable and to design stable control algorithms for them. Here, we outline Lyapunov stability, an important concept that can be used to study such systems.

Recall from Section 8.2 that we can represent a system using its state variables in the form of a state vector. The system's state space is the set of all values that can be assumed by its state vector. We declare the ideal system state to be its equilibrium, or relaxed, state when all the system state variables are at their nominal zero value: $\mathbf{x} = \mathbf{0}$. The goal of the controller, then, is to move the system from its current state to this ideal state: The path so taken over time is called the system's **trajectory**. The trajectory is much like the path taken by a marble as it rolls down a slope, except that the system's position at any time is described by a state vector rather than a scalar.

A system is said to be **stable in the sense of Lyapunov** if all system trajectories can be confined to a bounded part of the state space as long as the system was initially confined to some other part of the state space. In other words, if we are allowed to choose the initial conditions, a Lyapunov stable system never wanders too far from equilibrium.

More formally, consider a system described by:

$$\begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{f}(\boldsymbol{x}(t)) \\ \boldsymbol{f}(\boldsymbol{x}(t) = \mathbf{0}) &= \mathbf{0} \end{aligned} \qquad \text{(EQ 8.23)}$$

The first equation says that the system-state trajectory can be described in terms of a first-order differential equation. The second equation says that if the system is in equilibrium, then it continues to be in equilibrium (it is a fixed point of $\mathbf{f}$). This, of course, ignores disturbances.

Such a system is said to be **stable in the sense of Lyapunov** at $\boldsymbol{x} = \mathbf{0}$ if, for every real number $\varepsilon > 0$, there exists another real number $\delta > 0$ such that if $||\boldsymbol{x}(0)|| < \delta$ then for all $t \geq 0$ $||\boldsymbol{x}(t)|| < \varepsilon$, where $||\boldsymbol{x}|| = \sqrt{\boldsymbol{x}^T \boldsymbol{x}}$. So, as long as we can choose the initial conditions to be in a hypersphere centered at the origin and with radius δ, the system's trajectories will be confined to a hypersphere centered at the origin and with radius ε.

The system is **asymptotically stable** at $\boldsymbol{x} = \mathbf{0}$ if it is stable in the sense of Lyapunov, and, moreover, $\boldsymbol{x}(t) \to \mathbf{0}$ as $t \to \infty$. That is, we can choose initial conditions so that further evolution is not only bounded but also eventually converges to the equilibrium state.

Our overall goal is to prove that a nonlinear or time-variant system is either stable in the sense of Lyapunov or asymptotically stable. To do so, we define an auxiliary scalar function called the **Lyapunov function** $V(\boldsymbol{x})$. The Lyapunov function can be nearly arbitrarily defined and corresponds to the amount of energy in the system: The greater the energy, the larger the value of $V(\boldsymbol{x})$. We can think of a disturbance as adding energy to the system that is dissipated by the actions of the controller, so that the system eventually settles down to equilibrium. It can be shown that a system is stable in the sense of Lyapunov if it is possible to define a Lyapunov function for it such that the Lyapunov function decays over time (owing to the actions of the controller) along all system trajectories.

More formally, the Lyapunov stability theorem states that a system described by Equation 8.23 is stable in the sense of Lyapunov at $\boldsymbol{x} = \mathbf{0}$ if it is possible to define a scalar function $V(\boldsymbol{x}(t))$ such that

- $V(\boldsymbol{x}(t)=\mathbf{0}) = 0$
- $V(\boldsymbol{x}(t)) > 0;\ \boldsymbol{x}(t) \neq \mathbf{0}$
- $V(\boldsymbol{x}(t))$ is continuous and has continuous partial derivatives with respect to each component of $\boldsymbol{x}$
- $\frac{dV(\boldsymbol{x})}{dt} \leq 0$ for all system trajectories

Moreover, if $\dot{V}(\boldsymbol{x}) < 0$ (strictly less than zero) along all system trajectories, the system is asymptotically stable. This is also called **Lyapunov 2-stability**, or stability using the second method of Lyapunov.

The use of a Lyapunov function is both powerful and somewhat open-ended: If such a function can be found, even complex systems can be proved stable. On the other hand, finding such a function is a hit-or-miss process, other than for the simplest cases. In practice, we can use Lyapunov stability as a design tool by defining a plausible energy function and showing that control actions tend to always decrease system energy.

Model Predictive Control

S. Keshav
University of Cambridge

December 29, 2020

Abstract

This short note presents a conceptual overview of Model Predictive Control.

1 Introduction

The discussion in the text so far has been of classical (PID) and state-space control. These techniques work well when the system (or *plant*) is linear (or can be easily linearized in the neighbourhood of the operating point) and there is little coupling between the outputs, that is, the outputs are relatively independent of each other, depending only on the system state and plant inputs. However, many practical systems are non-linear and with coupled outputs. These systems are better controlled using *Model Predictive Control* (MPC).

At its heart, an MPC controller uses a *model* of the system to *predict* its expected evolution in response to its controlled and uncontrolled inputs. Specifically, the system is assumed to be fully described by its *state variables*. The model mathematically describes the relationship between the system's inputs and its outputs and the predictions are of the system state for the immediate future. Given these predictions, unlike PID controllers, whose controllers are relatively simple functions of the measured error, an MPC controller uses a potentially complex *optimizer* to choose control inputs such that:

- system *inputs* obey operating constraints
- system *outputs* are as close as possible to their reference values and obey operating constraints
- the *cost* of the control actions is minimized

Example 1: An AV Speed Controller We use a running example of a controller for an autonomous vehicle (AV). For simplicity, focus on a MPC-based speed controller, which controls how fast the AV drives using an accelerator and a braking system. The AV's state is its current speed. The model mathematically relates the accelerator setting and brake level (the controlled inputs) as well as environmental factors such as wind direction and speed and road slope (the uncontrolled inputs) to the expected speed of the car (the output). The controller uses this model to predict the future speed of the vehicle in response to both controlled and uncontrolled inputs and uses the prediction to choose a feasible accelerator and braking level such that (a) the vehicle speed matches the desired value and (b) changes in speed do not subject the vehicle to too high a G force. Ideally, this set of control decisions uses the least amount of fuel.

Importantly, to deal with unforeseen disturbances, the optimal *trajectory* of control inputs (i.e., the set of control inputs for the immediate future) is computed but not entirely carried out. Instead, it is periodically recomputed, and after each re-computation only the first few control actions are carried out; the remainder of the trajectory is ignored. Thus, if the anticipated system state diverges from the true measured state, previously computed control actions are ignored.

Example 2: Consider the actions of the AV speed controller when the AV turns a corner and spots a pedestrian on the road, an unanticipated situation. Prior to spotting the pedestrian, the controller would have chosen control inputs to ensure that the AV's speed matched the reference value. However, once the pedestrian is spotted, the prior control trajectory is set aside and a new control trajectory is computed to bring the AV to rest well before a potential collision.

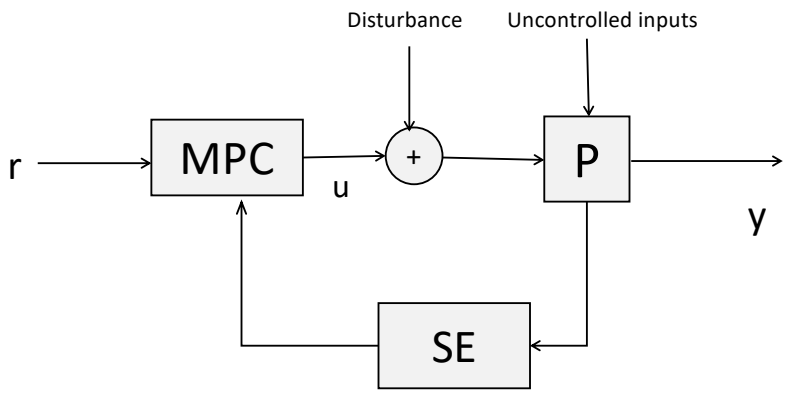


Figure 1: Conceptual model of a Model Predictive Controller (MPC)

Of course, this is wasteful in terms of computation overhead. Hence, an MPC designer must carefully decide how often to control the system, how much of the future trajectory to predict, and how best to model the system so that the optimal trajectory is easy to compute. We will discuss these issues later in this note.

2 Model

Let's make this intuitive description more precise. Consider the controller shown in Figure 1. Here, the reference vector **r** is one input to the **MPC** controller, the other input being the output of the state estimator **SE** that observes the plant **P** and estimates its current state. The output of the controller, **u**, is subject to disturbances before being given to the plant. The output of the plant is **y**, and our principal control goal is to ensure that $\mathbf{y} = \mathbf{r}$.

Example 3: For the case of the AV speed controller, the reference value is the scalar desired speed (this is assumed to be set by a higher-level controller). The plant is the AV, and its state corresponds to its speed. The state value is observed by the state estimator and made available to the controller. The output of the system is also its speed. Uncontrolled inputs to the plant include wind, road slope, pedestrians, barriers, and traffic signals. The accelerator and brake positions selected by the controller are the controlled inputs, and may themselves subject to disturbances; for example, the braking system may apply a braking force that slightly differs from the one intended.

We will assume that the MPC controller operates at discrete time steps, indexed by k, as shown in Figure 2. Each step is called a *sample time*. At the time marked *Now*, the MPC controller uses a model of the system to predict its anticipated future evolution. This requires a prediction of the future uncontrolled inputs over the *prediction horizon*. This prediction can use an arbitrarily complex algorithm; common techniques include Auto-Regressive Moving Average (ARMA) and its many variants, such as Seasonal ARMA, ARIMA, and ARMAX or predictors based on Deep Neural Networks. The prediction horizon should be long enough to take into account significant future events. In the example of the AV speed controller, this would include stop signs and traffic lights anticipated along its path.

Predictions are used to compute the trajectory of future control actions. We would like these actions to be *feasible* and *optimal*. That is, the actions must obey operating constraints, and the anticipated outputs must obey operating constraints, minimize control effort, and allow the output to closely track the reference value. To ensure optimality of the control actions, the MPC uses an *optimizer* to compute the optimal control trajectory over the course of a *control horizon*. The control horizon is typically shorter than the prediction horizon, since it is more difficult to compute, and, in any case, only the first step of the control trajectory is actually implemented.

Note that after the control actions are taken at the present time, time advances by one step, and the predictions and control actions are re-computed over the new prediction and control horizons. For this reason, MPC is also called *receding-horizon* control.

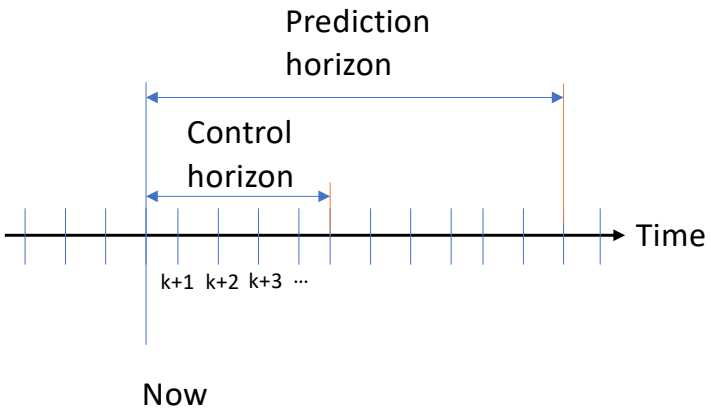


Figure 2: Time diagram

Example 4: For the AV speed controller, the MPC would use an optimizer to compute the accelerator and brake positions for the control horizon, taking into account the predictions of significant future events. However, at each time step, the controller would only actuate the first action in the trajectory.

In computing the control actions, the optimizer could be asked to ensure that the controlled input and outputs obey both *hard* and *soft* constraints. Hard constraints must always be met, whereas a soft constraint, when violated, only results in a penalty, with goal being to minimize this penalty.

Example 5: For the AV speed controller, a hard constraint is the maximum acceleration and the maximum braking level. These constraints must always be met. A soft constraint might be how often the brake level is changed: if this constraint were violated, the AV's passengers would feel the ride was jerky.

Note that the MPC controller does not require the plant or operating constraints to be linear. Instead the complexity of non-linear operation is dealt with by the optimizer and the predictor. The actions of the optimizer can be arbitrarily complex. They could, for instance, require the simulation of many possible future evolution paths of the system, with the best of these chosen as the one to realize. This can place a heavy computational burden on the controller.

In case of a highly non-linear system, predictions may be inaccurate and the system behaviour may be far from optimal. Moreover, the cost of computing the optimal trajectory may be quite high. Nevertheless, the overall control paradigm would still be applicable.

3 MPC parameters

This section discusses how one might choose MPC control parameters such as the sample time, prediction horizon, and control horizon.]https://www.youtube.com/watch?v=dAPRamI6k7Q

Sample time: The sample time determine how often the MPC controller takes control actions. The shorter this period, the more responsive the control. However, this also increases the cost of computation. A rule of thumb is to first measure the *rise time* of the uncontrolled system, that is, the time it takes to go from 10% to 90% of its final value, when given a step input. It is best to have the sample time be about 5-10% of the rise time, so that there is ample time to make corrections during the rise time.

Example 6: Consider the AV when it is given a step input, that is, the accelerator setting goes from zero to one unit (arbitrarily defined). We would expect that the speed of the AV would rise as shown in Figure 3. Then, we would like the sample time to be about 10-20 times smaller than this rise time.

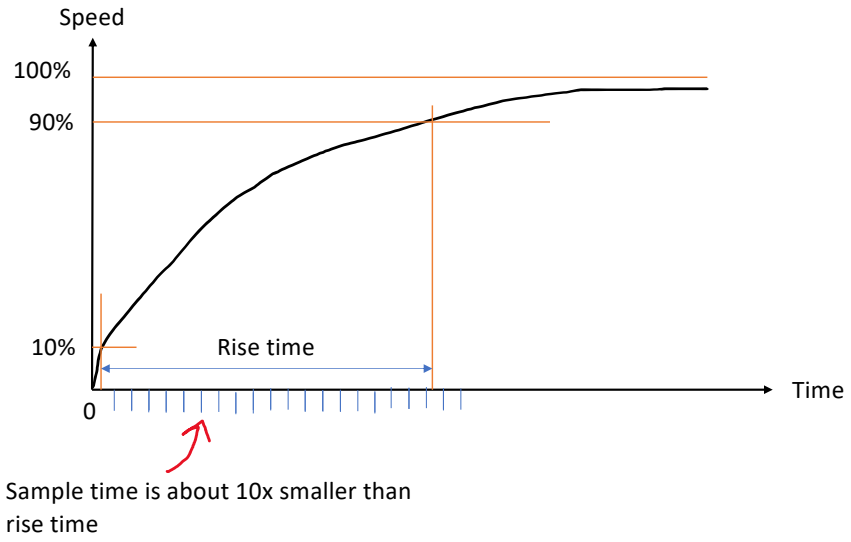


Figure 3: Guidance on choosing sample time.

Prediction horizon: How far should the controller predict into the future? Again, this balances computation time and accuracy of control. In general, the prediction horizon needs to be long enough to cover significant future events, such as an upcoming traffic light for an AV. From a control theoretic perspective, recall that when subjected to a unit step input a system exhibits both a *steady state* and *transient* response. The rule of thumb is that the prediction horizon should include about 20-30 sample times during the transient system response.

Control horizon: The control horizon is the time duration for which optimal control actions computed. One might think that the longer this horizon the better, but this makes the computation more expensive to carry out. Indeed, although the control horizon can be as long as prediction horizon, only the first few steps matter anyway, so there is no need to make the horizon too long. As a rule of thumb, the control horizon should be about 10%-20% of the prediction horizon.

Example 7: Imagine that an on-board map allows the AV to predict that a stop sign is expected, say, 100 time steps in the future. If the prediction horizon was greater than 100 time steps, this significant fact would be available to the MPC controller. Given this information, it should change the desired reference speed trajectory r to slow down the AV from its current speed to zero in 100 time steps. However, the control actions, i.e., the accelerator and brake settings, will need to be computed only for, say, the next 10 steps, which is the control horizon. The control horizon need not be longer since unexpected disturbances, such as wind gusts, may require a re-computation of control settings several times before the stop sign is reached.

Hard and soft constraints: Hard and soft constraints can be set on on both inputs and outputs. If a hard constraint cannot be met, the optimizer will be forced to give up, claiming a solution cannot be found. In this situation, there needs to be a fallback plan, otherwise the system will be uncontrolled! Thus, it is best to define most constraints to be soft, reserving hard constraints only to express safety or operational limits.

Example 8: The upper and lower limits of operation of the accelerator are hard constraints, as is the maximum braking force. On the other hand, the requirements that the speed not change too abruptly from one time step to another is a soft constraint. We can express the expected absolute change in speed as a *penalty variable* in the optimization objective, encouraging the optimizer to reduce this penalty to the smallest possible value.

If a pedestrian were to suddenly step in front of the AV, so that it could not stop in time even with the maximum braking force, then the optimizer would declare infeasibility and give up. A fallback plan, in this case, might be to maximally steer the car away from the pedestrian.

Optimization objective: The optimization objective is an expression or function that an optimizer tries to

minimize or maximize. As an MPC designer, a careful choice of the objective function allows the optimizer to balance between competing goals. Every objective function must include a term that reflects the error between the reference and actual output values, so that this objective is achieved. In addition, the objective function may have terms reflecting the amount of change in inputs or outputs between two control steps, as well as terms reflecting the cost of control (e.g., the amount of fuel used by the engine). We can balance between competing goals by assigning different weights to each goal, as the next example demonstrates.

Example 9: A simple objective function for the AV MPC controller that it would seek to minimize might be $\Sigma_k \alpha|r(k) - y(k)| + (1-\alpha)|(y(k) - y(k-1))|$, where α is a tuning parameter, $r(k)$ is the reference speed, and $y(k)$ is the speed at time step k. When α is close to 1, then the goal is to match the output speed to the reference as fast as possible (even if this results in a jerky ride), whereas when it is close to 0, the goal is to ensure a smooth ride. Intermediate values allow us to balance these goals. In practice, the objective function would need to far more complex to account, for example, for the cost of fuel.

Example 10: For completeness, here is an example of the optimization problem the MPC controller might solve:

$$\min_{acceleration(k), brake(k)} \sum_{k=now}^{k=k+controlhorizon} \alpha|r(k) - y(k)| + (1-\alpha)|(y(k) - y(k-1))| \tag{1}$$

$$y(k) = f(y(k-1), acceleration(k), brake(k)) \tag{2}$$

$$y(k) \geq 0 \tag{3}$$

$$acceleration(k) \geq 0 \tag{4}$$

$$brake(k) \geq 0 \tag{5}$$

where α is a tuning parameter, $r(k)$ is the reference speed, and $y(k)$ is the speed at time step k. f is the mathematical model that relates the prior state and current inputs to current outputs. Note that the objective function includes future time steps, thus requiring a predictor.

4 Some advanced concepts

This section discusses some advanced techniques for model predictive control.

4.1 Dealing with non-linearity

A significant advantage of MPC-based control over other approaches is that it can deal with non-linear systems. With MPC, any or all of the plant model, the constraints, and the control costs can be non-linear. However, this comes at the expense of additional computation. Moreover, there is no guarantee that the outcomes are optimal.

More specifically, with a linear plant model, linear constraints, and linear or quadratic costs, the optimizer has to solve a convex optimization problem, which is easy to solve, and the outcome is guaranteed to be optimal. However, if any of these are non-linear, then the optimization problem may be non-convex, which can lead to sub-optimal outcomes.

In some cases, the MPC approach can be modified to deal with non-linearities. Two variants of MPC that are widely used in practice to deal with non-linear system models are *adaptive* MPC and *gain-scheduled* MPC. In both approaches, the system linearized around its operating point, using a Taylor expansion, as discussed on page 327 of the text. If the system has multiple operating points, the output space is decomposed into a set of sub-spaces, and a linearized model is defined for each subspace. This results in a piecewise-linear approximation of the plant.

With *adaptive MPC* the piecewise-linear model is computed on the fly as the system changes. Note that with this approach, the structure of the optimization problem stays the same, only its parameters change to reflect the underlying subspace.

With *gain scheduled MPC*, the output space is linearized offline, and one controller is designed for each linearized model. As the operating point of the system evolves, a higher-level controller switches between the appropriate controllers. This uses more memory than the adaptive approach, but can be more responsive, since the linearization does not need to be computed on-the-fly.

4.2 Improving MPC performance

It should be clear by now that the main problem with the MPC approach is its high computational demand. However, there are some approaches to reducing this computational cost, as discussed next.

- The computational cost can be reduced by reducing the length of the prediction and control horizon. Generally speaking, prediction is easier to carry out than optimization. So, reducing the control horizon has the greater impact on reducing computational cost. However, especially with non-linear optimization, a reduction in computation may come at the expense of significant sub-optimality.
- In some cases it is possible to optimize over a longer control horizon without significantly increasing costs by using increasing larger time steps over the control horizon. For example, each time step can have a duration that multiplicatively increases the duration of the prior step. This allows the controller to respond to significant events quite far in the future, but with only a logarithmic increase in cost.
- A complex plant model is harder to simulate and optimize. If possible, portions of the plant model that are not strictly necessary should be excised. This is the intuition behind *model order reduction techniques*.
- If floating point operations are computationally expensive, then costs can be reduced by using lower precision or better data representation (as integers, for example, instead of floats).
- With *explicit MPC*, the optimal output of the controller for a given set of inputs is pre-computed and stored in a table. Then, in operation, instead of running the optimizer, the table can be looked up, reducing computational time. Of course, this requires substantial prior computation, as well as the need to store a potentially large lookup table corresponding to all possible operation states. As a compromise, the table could store (or cache) frequently occurring parameters combinations, reverting to standard operation when the system strays beyond the set of known states.

5 Conclusion

Model predictive control harnesses the power of modern microprocessors to compute optimal control actions based on the measured state. Unlike feedback or PID control, which does not work well when the plant model or constraints are non-linear, the MPC paradigm can deal with complex, coupled outputs and non-linearities. However, this comes at the cost of additional computation. Many variants of the basic approach have been developed to reduce this computational cost.

6 Further reading

Here are some references to read further about MPC. Rawlings[5] provides a more mathematical tutorial that links MPC to Linear Quadratic Gaussian control. Prediction using ARMA and its variants is discussed in Reference [3] and more recent approaches are surveyed in [2]. Alessio*et al* [1] survey approaches used for explicit MPC.

7 Acknowledgements

This note is based substantially on a 5-part tutorial from MATLAB on YouTube [4]. Thanks also to Fiodar Kazhamiaka for his comments on an earlier draft.

References

[1] Alessio, A., and Bemporad, A. A survey on explicit model predictive control. In *Nonlinear model predictive control*. Springer, 2009, pp. 345–369.

[2] Han, Z., Zhao, J., Leung, H., Ma, K. F., and Wang, W. A review of deep learning models for time series prediction. *IEEE Sensors Journal* (2019).

[3] Makhoul, J. Linear prediction: A tutorial review. *Proceedings of the IEEE 63*, 4 (1975), 561–580.

[4] MATLAB: Understanding Model Predictive Control. https://www.youtube.com/watch?v=8U0xiOkDcmw. Accessed: 2020-12-22.

[5] Rawlings, J. B. Tutorial overview of model predictive control. *IEEE control systems magazine 20*, 3 (2000), 38–52.

8.11 Partial Fraction Expansion

Partial fraction expansion allows a complex function $f(s)$ that is the ratio of two polynomials in the complex variable s to be represented in an alternative form to which one can easily apply the inverse Laplace transform. To begin with, let

$$f(s) = \frac{b_0 s^m + b_1 s^{m-1} + \ldots + b_m}{s^n + a_1 s^{n-1+\ldots+a_n}} = \frac{N(s)}{D(s)}$$

We first find the roots of $D(s)$ by solving the equation $D(s) = 0$, usually computed numerically by using such algorithms as the Jenkins-Traub algorithm. Let these roots be $\alpha_1, \alpha_2, \ldots, \alpha_n$. Then, we can write

$$f(s) = \frac{N(s)}{(s-\alpha_1)(s-\alpha_2)\ldots(s-\alpha_n)}$$

The next steps depend on the nature of the roots. There are three cases: distinct roots, complex conjugate roots, and repeated roots.

8.11.1 Distinct Roots

In this case, we write

$$f(s) = \frac{a_1}{(s-\alpha_1)} + \frac{a_2}{(s-\alpha_2)} + \ldots + \frac{a_n}{(s-\alpha_n)}$$

where $a_i \in C$. The coefficient a_i is given by

$$a_i = \lim_{s \to \alpha_i} (s-\alpha_i) f(s)$$

EXAMPLE 8.20: DISTINCT ROOTS

Find the partial fraction expansion of the polynomial fraction $\frac{1}{s^2+3s+2}$.

Solution:

We write the fraction as

$$f(s) = \frac{1}{(s+1)(s+2)} = \frac{a_1}{s+1} + \frac{a_2}{s+2}.$$

Then,

$$a_1 = \lim_{s \to -1} (s+1)\left(\frac{1}{(s+1)(s+2)}\right) = \frac{1}{-1+2} = 1$$

and

$$a_2 = \lim_{s \to -2} (s+2)\left(\frac{1}{(s+1)(s+2)}\right) = \frac{1}{-2+1} = -1,$$

so that

$$f(s) = \frac{1}{s+1} - \frac{1}{s+2}.$$

8.11.2 Complex Conjugate Roots

Suppose $D(s)$ has one pair of complex conjugate roots: They must be conjugate if the coefficients of $D(s)$ are real. Then, we write

$$f(s) = \frac{a_1}{(s-(\alpha_1 + j\beta_1))} + \frac{a_2}{(s-(\alpha_1 - j\beta_1))} + \ldots + \frac{a_n}{(s-\alpha_n)}$$

It can be shown that a_1 and a_2 are the complex conjugates of each other. Moreover,

$$a_1 = \lim_{s \to (\alpha_1 + j\beta_1)} (s-(\alpha_1 + j\beta_1)) f(s).$$

EXAMPLE 8.21: COMPLEX CONJUGATE ROOTS

Find the partial fraction expansion of the polynomial fraction $\frac{1}{s^2 - 6s + 25}$.

Solution:

We write the fraction as

$$f(s) = \frac{1}{(s-(3+j4))(s-(3-j4))} = \frac{a_1}{(s-(3+j4))} + \frac{a_2}{(s-(3-j4))}.$$

Then,

$$a_1 = \lim_{s \to (3+j4)} (s-(3+j4))\left(\frac{1}{(s-(3+j4))(s-(3-j4))}\right) = \frac{1}{(3+j4-(3-j4))} = \frac{1}{j8}$$

and $a_2 = \frac{1}{-j8}$, so that $f(s) = \frac{1}{j8(s-(3+j4))} - \frac{1}{j8(s-(3-j4))}$.

8.11.3 Repeated Roots

If $D(s)$ has repeated roots, the partial fraction expansion is somewhat more complex. Suppose that $D(s)$ has r repeated roots α_1 and that the other roots are distinct. Then,

$$f(s) = \frac{a_{1(r)}}{(s-\alpha_1)^r} + \frac{a_{1(r-1)}}{(s-\alpha_1)^{r-1}} + \ldots + \frac{a_{1(1)}}{(s-\alpha_1)} + \frac{a_2}{(s-\alpha_2)} + \ldots + \frac{a_n}{(s-\alpha_n)}$$

where the ith repeated root, $i = 0,1,\ldots, r-1$ is given by

$$a_{1(r-i)} = \lim_{s\to\alpha_1}\left(\frac{1}{i!}\frac{d^i}{ds^i}((s-\alpha_1)^r f(s))\right)$$

EXAMPLE 8.22: REPEATED ROOTS

Find the partial fraction expansion of the polynomial fraction $\frac{1}{s^3+7s^2+16s+12}$.

Solution:

We write the fraction as $f(s) = \frac{1}{(s+2)^2(s+3)} = \frac{a_{1(2)}}{(s+2)^2} + \frac{a_{1(1)}}{s+2} + \frac{a_2}{s+3}$. Then,

$$a_{1(2-0)} = \lim_{s\to -2}\left(\frac{1}{0!}\frac{d^0}{ds^0}((s+2)^2 f(s))\right) = \lim_{s\to -2}\left(\frac{1}{s+3}\right) = 1$$

$$a_{1(2-1)} = \lim_{s\to -2}\left(\frac{1}{1!}\frac{d^1}{ds^1}((s+2)^2 f(s))\right) = \lim_{s\to -2}\left(\frac{d}{ds}\left(\frac{1}{(s+3)}\right)\right)$$

$$= \lim_{s\to -2}\left(\frac{s+3\frac{d}{ds}1 - 1\frac{d}{ds}s+3}{s+3^2}\right) = -1$$

$$a_2 = \lim_{s\to -3}(s+3)f(s) = \lim_{s\to -3}\left(\frac{1}{(s+2)^2}\right) = 1$$

$$f(s) = \frac{1}{(s+2)^2} - \frac{1}{s+2} + \frac{1}{s+3}.$$

8.12 Further Reading

There are many excellent texts on control theory. A standard reference is B. C. Kuo and F. Golnaraghi, *Automatic Control Systems*, Wiley, 2002. Classical control is discussed in detail with numerous examples in M. Gopal, *Control Systems: Principles and Design*, McGraw-Hill, 2008. The standard reference for digital control systems is K. Ogata, *Discrete-Time Control Systems*, 2d ed., Prentice Hall, 1995.

8.13 Exercises

1. A bandwidth management system

Consider the following problem: An ISP wants to ensure that, on a specific congested link, the fraction of P2P traffic is never more than r percent. If this fraction is exceeded, the ISP sends TCP reset packets to P2P connection endpoints, terminating them, and thus reducing their load. For this system, identify the plant, the command, the control input, the disturbance, and the output.

2. Effort and flow

What are the effort and flow variables in the system of Exercise 1?

3. State space representation

Give a state space representation for the system of Exercise 1. Assuming that the control rule is to reduce the number of P2P connections by u connections over a time period T if the current P2P traffic fraction exceeds r percent, represent this in the state-evolution equation. Note that the system is subject to a random disturbance at the instantaneous rate of w.

4. Transfer function

What is the transfer function for the system of Exercise 1 in the regime when the congested link has more than r percent of P2P connections?

5. First-order system

When given a step input, a first-order system reaches 63% of its asymptotic value of 4.25 units at time $3s$. What is its transfer function?

6. Second-order system

Prove that for a critically damped second-order system,

$$y(t) = K[1 - e^{-\omega_n t} - \omega_n t e^{-\omega_n t}].$$

7. Proportional-mode control

What is the relationship between the loop gain and the system pole for the proportional-mode controller discussed in Example 8.9?

8. Integral-mode control

What is the impulse response of the integral-mode controller in Example 8.10?

9. Stability

Use a numerical solver to determine the stability of an LTI system with denominator polynomial $3s^5 - 4s^3 + 7s^2 + 10s + 1$.

10. Matrix exponential

Prove that $e^{\boldsymbol{A}t} = \boldsymbol{I} + \boldsymbol{A}t + \frac{\boldsymbol{A}^2 t^2}{2!} + \frac{\boldsymbol{A}^3 t^3}{3!} + \ldots$ satisfies $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$.

11. Matrix exponential

Show that for a diagonal matrix $e^{\boldsymbol{A}t} = \begin{bmatrix} e^{a_{11}t} & 0 & 0 & 0 \\ 0 & e^{a_{22}t} & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & e^{a_{nnt}} \end{bmatrix}$.

Use this to compute $e^{\boldsymbol{A}t}$ for $\boldsymbol{A} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & -4 & 0 \\ 0 & 0 & -1 \end{bmatrix}$.

12. Partial fraction expansion

Find the partial fraction expansion of $\frac{s}{(s+3)(s+5)}$.

13. Partial fraction expansion

Find the partial fraction expansion of the polynomial fraction $\frac{1}{s^2 - 4s + 29}$.